


# User Manual

Version 2.8-rc2

A music tracker for the POKEY sound chip
of the Atari 8-bit computers (XL/XE)

Based on RASTER Music Tracker by Radek Štěrba (R.I.P.), VinsCool and Peter Dell (JAC!)

November 2026

# About this manual

This manual describes RITMO, a music tracker for the POKEY sound chip of the Atari XL/XE 8-bit computers. RITMO (*RITMO Is a Tracker for Music On Atari*) is a fork of the RASTER Music Tracker (RMT), and runs on Linux, Windows and macOS.

It is written for two kinds of reader: the musician who wants to compose, and the Atari programmer who wants to use the result in a demo or a game. Chapters 1 to 5 introduce the program and walk through a first song; the next ones describe every part of the window, the menus, the dialogs, the import and export formats, MIDI input and the scripting; the appendices are the complete reference of the keys, the fields and the file format.

## Conventions

- Keys are written like `CONTROL+S` or `F5`. `CONTROL` is the `Ctrl` key (on macOS the `Control` key, not `Command`).
- A menu item is written **File** → **Open...**.
- Hexadecimal numbers have a `$` in front: `$3F` is 63. The tracker shows its numbers in hexadecimal: line numbers, instruments, volumes.
- The names of the fields of the screen are written as the program writes them, in capital letters: `VOLUME`, `TABLE LENGTH`.
- The screenshots are taken from the program itself, by the script `doc/manual/make-screenshots.sh`, with the default settings of a first start and the same font on every computer.

**RMT and RITMO in the reference**

The reference tables are converted from the documents of the original RMT, and still call the program RMT in places. RMT is also the name of the file format and of the player routines: an *RMT module* and the *RMT drivers* keep their names in RITMO.

**Where the reference tables come from**

The tables of the menu commands, of the note keys, of the scripting commands and of the tracker drivers (appendix A and chapters 15 and 16) are converted from the Markdown documents in the `doc/` folder of the program, and the menu and note key tables of those are written by the program itself (the `dump actions` and `dump notekeys` script commands). They are always the state of the version this manual is built from.

# Contents


**About this manual** i

**1 Introduction** 1

1.1 What RITMO is . . . 1
1.2 History . . . 1
1.3 What is different from RMT . . . 2
1.4 Files used by RITMO . . . 2

**2 Installing and starting** 3

2.1 The packages . . . 3
2.2 Linux, architecture by architecture . . . 5
2.2.1 x86_64 and aarch64 (Raspberry Pi): the AppImage . . . 5
2.2.2 riscv64, ppc64 and ppc64le: the AppImage . . . 5
2.2.3 riscv64, ppc64 and ppc64le: the .deb . . . 6
2.2.4 riscv64, ppc64 and ppc64le: the archive . . . 6
2.3 Windows and macOS in detail . . . 6
2.3.1 Windows 64 bit: the installer . . . 6
2.3.2 Windows 64 bit: the .zip . . . 7
2.3.3 macOS 12 or later: the disk image . . . 7
2.4 Starting the program . . . 7
2.5 The window . . . 7
2.6 Sound . . . 8
2.6.1 When the sound ignores the desktop . . . 8
2.7 Where the settings are kept . . . 9

**3 Concepts** 10

3.1 Time . . . 10
3.2 The sound of the POKEY . . . 11

**4 The main window** 12

4.1 The areas of the screen . . . 12
4.2 The four editing parts . . . 13
4.3 Edit mode and prove mode . . . 15

4.4 The menus and the toolbars . . . 15
4.5 The mouse . . . 15

**5 Your first song** **16**
5.1 A new song . . . 16
5.2 Entering notes . . . 16
5.3 Choosing the instrument . . . 17
5.4 Playing . . . 17
5.5 More tracks and song lines . . . 17
5.6 Saving and exporting . . . 17

**6 Editing tracks** **18**
6.1 The cursor . . . 18
6.2 The length of a track . . . 18
6.3 Playing while editing . . . 18
6.4 Blocks . . . 19
6.4.1 The Effects and Tools dialog . . . 19
6.5 Tracks as objects . . . 19
6.6 Undo . . . 20

**7 Instruments** **21**
7.1 The parts of an instrument . . . 21
7.2 Editing . . . 21
7.3 Instrument files and the **Instrument** menu . . . 22
7.4 The distortions . . . 22
7.5 The instrument active help . . . 22

**8 The song** **23**
8.1 Song lines . . . 23
8.2 Subsongs . . . 23
8.3 Bookmarks . . . 23
8.4 The info area . . . 24
8.5 Changing the kind of song . . . 24
8.6 Optimizing the size . . . 25

**9 Playing and monitoring** **26**
9.1 Play commands . . . 26
9.2 Channels . . . 26

9.3 Watching the sound . . . 26
9.4 Playing from the keyboard . . . 27

**10 The menus** **28**
10.1 File . . . 28
10.2 Edit . . . 28
10.3 View . . . 29
10.4 Play . . . 29
10.5 Channels . . . 29
10.6 Song . . . 30
10.7 Instrument . . . 30
10.8 Track . . . 31
10.9 Block . . . 31
10.10Pokey . . . 32
10.11Tools . . . 32
10.12Help . . . 32

**11 Options and tuning** **33**
11.1 The Options dialog . . . 33
11.1.1 General . . . 33
11.1.2 Keyboard . . . 34
11.1.3 MIDI . . . 34
11.1.4 Paths . . . 34
11.2 The module properties . . . 35
11.3 The tuning . . . 35

**12 Import and export** **37**
12.1 Importing . . . 37
12.2 Exporting . . . 38
12.2.1 RMT stripped . . . 39
12.2.2 ASM simple notation . . . 40
12.2.3 SAP, SAP-R and LZSS . . . 40
12.2.4 XEX . . . 41
12.2.5 XEX with the RMT player . . . 41
12.2.6 Relocatable ASM . . . 43
12.2.7 The RMT player in a C program (cc65) . . . 43
12.2.8 WAV . . . 44

12.3 Exporting from the command line . . . 44

**13 MIDI input** **45**

13.1 Options . . . 45
13.2 The MIDI channels . . . 46
13.3 Scripts . . . 46

**14 The Pokey explorer** **47**

14.1 The keys . . . 47

**15 Tracker drivers** **49**

15.1 Tracker Drivers . . . 49
15.2 Tracker Driver Versions . . . 49
15.2.1 RMT 1.28 Unpatched by Raster . . . 50
15.2.2 RMT 1.28 Unpatched with Tuning . . . 50
15.2.3 RMT 1.25 Patch3 Instrumentarium by Analmux . . . 50
15.2.4 RMT 1.27 Patch6 by Analmux . . . 50
15.2.5 RMT 1.28 Patch8 by Analmux . . . 51
15.2.6 RMT 1.34 Patch16 by VinsCool . . . 51
15.2.7 RMT 1.28 Patch Prince of Persia by VinsCool . . . 52

**16 Scripting** **53**

16.1 Running a script . . . 53
16.2 Syntax . . . 53
16.3 Commands . . . 54
16.4 Export formats and options . . . 55
16.5 Example . . . 56
16.6 Environment variables . . . 57
16.7 Tests . . . 57
16.8 Not available as script commands . . . 57

**A Reference of the commands, keys and fields** **58**

A.1 Note Keys . . . 65
A.1.1 QWERTY . . . 65
A.1.2 QWERTZ . . . 66
A.1.3 AZERTY . . . 67
A.2 For All Edit Modes . . . 68
A.2.1 Hotkeys . . . 68

A.3 TRACK EDIT . . . 70
A.3.1 Fields . . . 70
A.3.2 Hotkeys . . . 70
A.3.3 Note Column . . . 71
A.3.4 Instrument Number Column . . . 72
A.3.5 Volume Column . . . 72
A.3.6 Speed Column . . . 72
A.3.7 Block Operations . . . 72
A.3.8 Block Data Modifications . . . 73
A.3.9 Mouse control . . . 74
A.3.10 Tracks top area (Shown as TRACK L1 to TRACK R4 above pattern tracks) 74
A.4 INSTRUMENT EDIT . . . 74
A.4.1 Fields . . . 74
A.4.2 Table of Notes/Frequencies . . . 75
A.4.3 Envelope . . . 75
A.4.4 Sound Type (DISTORTION Parameter) . . . 75
A.4.5 Envelope command (COMMAND parameter) . . . 76
A.4.6 Hotkeys . . . 76
A.4.7 Table of notes/frequencies . . . 77
A.4.8 Envelope Parameters . . . 77
A.4.9 Mouse control . . . 78
A.4.10 Volume Envelope Area . . . 78
A.5 INFO EDIT . . . 78
A.5.1 Hotkeys . . . 78
A.5.2 Mouse control . . . 79
A.5.3 MAXTRACKLENGTH, MONO/STEREO, PAL/NTSC . . . 79
A.6 SONG EDIT . . . 79
A.7 PROVE (JAM) MODE . . . 79

**B The RMT module file format 80**

B.0.1 Segment Binary Load Header (Mandatory) . . . 80
B.0.2 Segment Names (Optional) . . . 80
B.0.3 Module Header Struct . . . 80
B.0.4 Order of the Structures . . . 81
B.0.5 Instrument Struct . . . 81
B.0.6 Table of Notes Struct . . . 82

B.0.7 Envelope Struct . . . 82
B.0.8 Track Struct . . . 82
B.0.9 Instruments Table . . . 83
B.0.10 Track Addresses Table . . . 83
B.0.11 Track Sequence Table . . . 84

**C MIDI implementation chart** **85**

**D Building from the source** **86**
D.1 Linux . . . 86
D.2 Windows and macOS . . . 86
D.3 Linux riscv64 and ppc64 . . . 86
D.4 Core only and tests . . . 87
D.5 Test hooks . . . 87

**E Credits and licenses** **88**
E.1 Credits . . . 88
E.2 The fonts . . . 88
E.3 Disclaimer . . . 88

CHAPTER 1

# Introduction

## 1.1 What RITMO is

The Atari XL/XE computers make their sound with the POKEY chip: four independent voices (eight when two chips are used, as in many stereo songs), each with a frequency divider, a choice of noise and tone distortions and a volume. A *tracker* is the kind of music editor that was born on the Amiga: the music is a grid of numbers, one column for each voice and one row for each beat, and the notes are typed with the computer keyboard as on a piano.

RITMO is such a tracker for the POKEY. A song made with it can be played on a real Atari: the program produces the same data the player routines of the Atari read, and exports them in several formats (a SAP file for the Atari players, an executable `.xex`, assembler source, the compact RMT module for your own program). The sound you hear while composing comes from an emulation of the Atari's 6502 processor and of the POKEY chip built into the program, running the same player routine, so it is what the Atari will play.

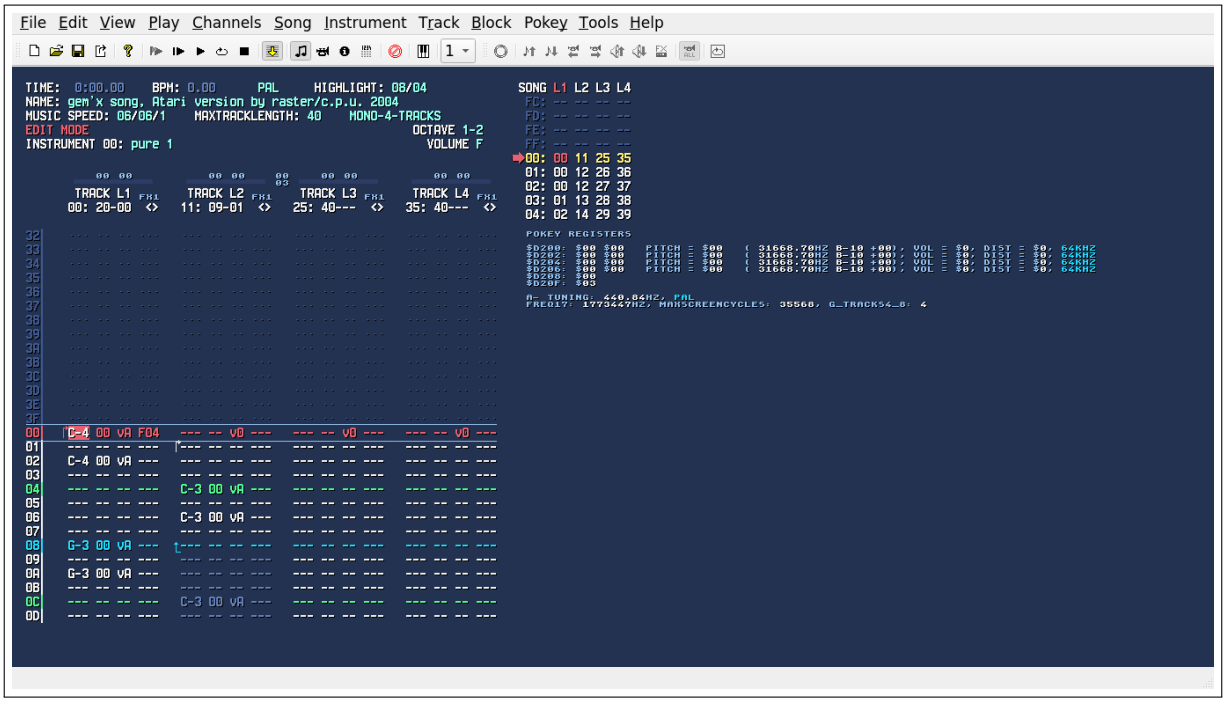



**Figure 1.1.** The main window of RITMO with a song loaded.

## 1.2 History

RITMO is a fork of the *RASTER Music Tracker*, RMT. The first version, 1.0, was written by Radek Štěrba (Raster/C.P.U.) in 2002; his last version is 1.28, of 2009. It was a small revolution for the Atari musicians: the first tracker for the POKEY with a graphical envelope editor, and the one

nearly everybody used. Radek Štěrba has since passed away; this manual is, among other things, a way to remember his work. R.I.P.

Vin Samuel (*VinsCool*) continued the program from 2021 to 2024, up to the version 1.34: a new player routine with more tuning tables, a tuning calculator, an automatic high pass filter, the SAP and XEX export and many corrections. Peter Dell (*JAC!*) carries on the original project from 2024 (version 1.35 and the planned version 2) and made it possible for this fork to exist: everything described here is built on his work.

The RITMO fork keeps the code of the tracker, the file formats and the player routines of RMT 1.35, and replaces what tied it to Windows. It is renamed so that it cannot be confused with the original RMT, and so that the Windows-only (MFC) parts of the code could be removed.

## 1.3 What is different from RMT

- The program is written with Qt: it runs natively on Linux, on Windows and on macOS (Intel and Apple Silicon). The look of the tracker screen is the original one, pixel for pixel.
- The 6502 and the POKEY emulations are built in: the `sa_c6502.dll` and `apokeysnd.dll` libraries of the Windows versions are not needed any more.
- The sound goes out through PortAudio, and the MIDI input comes in through RtMidi.
- The configuration is kept per user in the settings of the system, not in an `.ini` file next to the program. The settings of RMT are taken over at the first start.
- Songs, instruments and tracks are the same files as in RMT 1.3x: you can move your work between the programs. The files keep their extensions, `.rmt` and the others.

## 1.4 Files used by RITMO

| Extension | Content |
|---|---|
| `.rmt` | an RMT module: the song, its tracks and its instruments (and, if the song was saved with **Save**, the names of the song and of the instruments) |
| `.txt` | the same song as a text file, which can be read and edited with any editor; a single track is saved as a text file too |
| `.rmw` | a song work file: an RMT module with the state of the editing |
| `.rti` | a single instrument |
| `.sap`, `.sapr`, `.xex`, `.asm`, `.wav`, `.lzss` | the exported formats (chapter 12) |
| `.mod`, `.tmc`, `.tm8` | the formats that can be imported |
| `.rmtscript` | a script (chapter 16) |

CHAPTER 2

# Installing and starting

## 2.1 The packages

Everything the program needs is inside each package: the tracker drivers and the other files it loads at run time are in the `resources` folder next to the program. (The `.deb` of riscv64, ppc64 and ppc64le asks the package manager for the libraries instead of carrying them, section 2.2.)

| System | Package | How to start it |
|---|---|---|
| Linux x86_64 (Debian 11, Ubuntu 20.04 and newer) | `Ritmo-`*version*`-x86_64.AppImage` | make it executable (`chmod +x`) and run it |
| Windows 64 bit | `Ritmo-Windows-x64-Setup-`*version*`.exe`, `Ritmo-Windows-x64-`*version*`.zip` | run the installer (it asks whether to install for all the users or for the current one, and offers a desktop icon and the `.rmt` file type), or unzip the archive anywhere and run `Ritmo.exe` (the program is not signed: Windows may ask to confirm) |
| macOS 12 or later, Apple Silicon or Intel | `Ritmo-macOS-arm64-`*version*`.dmg`, `Ritmo-macOS-x86_64-`*version*`.dmg` | drag `Ritmo.app` to Applications; the first time open it with the right button and **Open** (the program is not signed) |
| Linux aarch64: Raspberry Pi 3/4/5 with a 64 bit system (Raspberry Pi OS 13, Debian 13, Ubuntu 24.04 and newer) | `Ritmo-`*version*`-aarch64.AppImage` | as the x86_64 one |
| Linux riscv64 (Debian 13 and newer) | `Ritmo-`*version*`-riscv64.AppImage`, `ritmo_`*version*`_riscv64.deb`, `Ritmo-Linux-riscv64-`*version*`.tar.gz` | run the AppImage, install the `.deb` with `apt`, or unpack the archive (section 2.2) |
| Linux ppc64: PowerPC 64 bit big endian (Debian ports, *sid*) | `Ritmo-`*version*`-ppc64.AppImage`, `ritmo_`*version*`_ppc64.deb`, `Ritmo-Linux-ppc64-`*version*`.tar.gz` | as riscv64 |
| Linux ppc64le: PowerPC 64 bit little endian, POWER8 and newer (Debian 13 and newer) | `Ritmo-`*version*`-ppc64le.AppImage`, `ritmo_`*version*`_ppc64el.deb`, `Ritmo-Linux-ppc64el-`*version*`.tar.gz` | as riscv64 (from 2.6) |

Every release also has `Ritmo-Songs-and-Instruments-`*version*`.zip`: the example songs (`.rmt`) and instruments (`.rti`), which the packages of the program do not carry (`songs` and `instruments` of the `rmt` folder of the repository). Unpack it anywhere, for example in your home or Documents folder: the file dialogs of RITMO remember the last folder.

RITMO can also be built from the source with CMake and Qt (appendix D).

**Note**

The packages of the versions up to 2.2.1 were called `RMT-*` and the program `rmt` (`Rmt.exe`). From the next releases the packages are `Ritmo-*` and the program `ritmo` (`Ritmo.exe`). From 2.6 their names carry the version, so that several can be kept side by side: *version* is `2.6` for the release 2.6, `2.6-rc3` for a test build; up to 2.5 the names had no version (`Ritmo-Linux-x86_64.AppImage`); from 2.6 the AppImage has no `Linux` in its name (`Ritmo-2.6-x86_64.AppImage`) and can update itself with AppImageUpdate. The examples of this chapter use `2.6`.

## 2.2 Linux, architecture by architecture

`uname -m` tells the architecture of the machine: `x86_64`, `aarch64`, `riscv64`, `ppc64` or `ppc64le` (whose Debian name, in the `.deb` and the `.tar.gz`, is `ppc64el`).

### 2.2.1 x86_64 and aarch64 (Raspberry Pi): the AppImage

An AppImage is one file with the program and everything it needs (Qt, PortAudio, RtMidi). It only needs the `libfuse2` library to start (on Debian 13 and Ubuntu 24.04 the package is called `libfuse2t64`):

```
sudo apt install libfuse2                  # libfuse2t64 on Debian 13 and Ubuntu 24.04
chmod +x Ritmo-2.6-x86_64.AppImage   # Raspberry Pi: Ritmo-2.6-aarch64.AppImage
./Ritmo-2.6-x86_64.AppImage mysong.rmt
```

Without FUSE the file can still run, unpacking itself in a temporary folder:

```
./Ritmo-2.6-x86_64.AppImage --appimage-extract-and-run mysong.rmt
```

The x86_64 AppImage runs on Debian 11, Ubuntu 20.04 and newer. The aarch64 one needs glibc 2.38 or newer: Raspberry Pi OS 13 (*trixie*), Debian 13, Ubuntu 24.04 and newer, on a Raspberry Pi 3, 4 or 5 with a 64 bit system (`uname -m` must say `aarch64`: on a 32 bit system it says `armv7l` and there is no package).

### 2.2.2 riscv64, ppc64 and ppc64le: the AppImage

From 2.6 these architectures have an AppImage too, started as the x86_64 one:

```
sudo apt install fuse3                     # FUSE for the AppImage
chmod +x Ritmo-2.6-riscv64.AppImage        # Ritmo-2.6-ppc64.AppImage, Ritmo-2.6-ppc64le.
    AppImage
./Ritmo-2.6-riscv64.AppImage mysong.rmt
./Ritmo-2.6-riscv64.AppImage --appimage-extract-and-run mysong.rmt   # without FUSE
```

It carries the same files as the archive below: Qt 6, PortAudio and RtMidi, but not glibc and the graphics and sound libraries of the drivers, so it needs the same Debian releases and the packages that the `README.txt` of the archive lists. There is no official `appimagetool` for these architectures yet: the AppImages are made with its ports to riscv64, ppc64 and ppc64le and to their runtimes (https://github.com/gianlucarenzi/appimagetool, https://github.com/gianlucarenzi/type2-runtime), proposed to the AppImage project.

### 2.2.3 riscv64, ppc64 and ppc64le: the .deb

For Debian and the systems derived from it, riscv64 and ppc64le (PowerPC 64 bit *little endian*, POWER8 and newer) need Debian 13 or newer, and ppc64 (PowerPC 64 bit *big endian*) the *sid* release of Debian ports, the only one that has this architecture. The `.deb` contains the program and `apt` installs the libraries it needs (Qt 6, PortAudio, RtMidi...) by itself:

```
sudo apt install ./ritmo_2.6_riscv64.deb   # ./ritmo_2.6_ppc64.deb, ./ritmo_2.6_ppc64el.deb
ritmo mysong.rmt                           # also in the menu of the desktop
```

The `./` before the name is needed: without it `apt` looks for a package of that name in the repositories. The program goes in `/usr/lib/ritmo`, with the link `/usr/bin/ritmo`. To remove it:

```
sudo apt remove ritmo
```

### 2.2.4 riscv64, ppc64 and ppc64le: the archive

For the other distributions, or where `apt` is not used, the `.tar.gz` carries Qt 6, PortAudio and RtMidi (folders `lib` and `plugins`); of the machine it only uses glibc and the graphics (OpenGL) and sound (ALSA) libraries of its drivers. The `README.txt` inside lists the Debian packages for these:

```
tar xzf Ritmo-Linux-riscv64-2.6.tar.gz     # Ritmo-Linux-ppc64-2.6.tar.gz, Ritmo-Linux-ppc64el
    -2.6.tar.gz
cd Ritmo-Linux-riscv64                     # the folder has no version
cat README.txt                             # the packages the machine needs
sudo apt install <the packages of README.txt>
./ritmo mysong.rmt
```

`ritmo` is a script that sets where the libraries and the plug-ins are: it can be started from any folder, and the `resources` folder must stay next to it. Nothing is installed: to remove it, delete the folder. The glibc of the machine cannot be older than the one the archive was built with (Debian 13 for riscv64 and ppc64le, *sid* for ppc64).

**Note**

There are no packages for PowerPC 32 bit. The packages of riscv64, ppc64 and ppc64le have been built and started (without a display) in an emulator, not on real machines; the engine of the ppc64 build gives the same results as the x86_64 one on all the songs of the distribution. If you try them, please tell the result in the issues of the project.

## 2.3 Windows and macOS in detail

### 2.3.1 Windows 64 bit: the installer

The installer is `Ritmo-Windows-x64-Setup-`*version*`.exe`. The program is not signed, so when it runs, Windows (SmartScreen) may ask to confirm: choose **More info** and then **Run anyway**. The installer asks whether to install for all the users (it needs the administrator) or only for the current one, and offers a desktop icon and the association of the `.rmt` files with RITMO, so that a double click on a song opens it. It adds an entry to the Start menu and to the list of the programs of Windows, from which it is removed. The settings are kept in the registry of the user, so a new installation does not lose them.

### 2.3.2 Windows 64 bit: the .zip

`Ritmo-Windows-x64-`*version*`.zip` has the same files without the installer: unzip it in any folder (also on a USB stick) and run `Ritmo.exe`; the `resources` folder and the DLLs of Qt must stay next to it. Nothing is installed and no file type is associated: to remove it, delete the folder (the settings stay in the registry, `HKEY_CURRENT_USER\Software\ritmo-atari.org`).

### 2.3.3 macOS 12 or later: the disk image

Open `Ritmo-macOS-arm64-`*version*`.dmg` (Apple Silicon) or `Ritmo-macOS-x86_64-`*version*`.dmg` (Intel; it also runs on Apple Silicon through Rosetta 2) and drag `Ritmo.app` to the `Applications` folder shown in the window. The application is signed only ad hoc, not notarized: the first time, open it with the right button (or Control and click) and **Open**, then confirm. After that it starts normally. It carries Qt and the libraries, nothing else needs to be installed. To remove it, move `Ritmo.app` to the bin; the settings are in `~/Library/Preferences/org.ritmo-atari.ritmo.plist`.

## 2.4 Starting the program

Start RITMO like any other program. The first start opens the window with an empty song. The program accepts a song on the command line, which is opened at the start:

```
ritmo mysong.rmt
```

With the option `/SCRIPT:` the program runs a script without showing its window (chapter 16):

```
ritmo /SCRIPT:build.rmtscript
```

With `--scale=`*n* (also `/scale:`*n*), from 100 to 300, the program starts with that interface size (section 2.5) for this session only: the settings are not changed.

```
ritmo --scale=200 mysong.rmt
```

## 2.5 The window

The window is made for a laptop with a 1366×768 screen: that is its size the first time the program starts (reduced to what the screen has, if the screen is smaller), and the whole screen of a mono song fits in it, with the registers of the POKEY next to the song area (**Pokey** → **Pokey Chip Registers**, on by default). A stereo song also fits, with its eight tracks and the song area, but the registers of the POKEY, which need about 650 more pixels on the right, are left out: to see them, make the window wider (at least about 1720 pixels) or use a larger screen. After that the program remembers the size and the position of the window and opens it the same way next time (if the screen is smaller than before, the window is reduced to fit).

The *interface size* (section 11.1) is chosen the first time too, from the size of the screen: 100 % on the screens up to Full HD, 200 % on a screen of 2560×1440, 300 % on a 4K screen with no scaling of the system. Only whole multiples are chosen, so that the pixels of the tracker stay square. The choice can be changed in the **Options** dialog, and the settings taken over from RMT keep the size they had.

## 2.6 Sound

The sound goes out through PortAudio, to the default output device of the system. On Linux, when a PulseAudio or PipeWire sound server runs, RITMO is one of its clients like any other program: it plays on the output chosen in the desktop, follows it when it changes (Bluetooth headphones, for example) and has its own volume in the mixer. It plays on the `default` device of ALSA when that one goes to the sound server (the `pipewire-alsa` package, or the `pulse` plugin of ALSA). When it does not, the AppImage talks to the sound server directly (its PortAudio has PulseAudio support, which PipeWire understands too); the `.deb` and `.tar.gz` packages, whose PortAudio has not, and every package without a sound server play on the `default` device of ALSA, which goes where the configuration of ALSA says. The `RMT_AUDIO_HOST_API` environment variable (`ALSA`, `JACK`, `PulseAudio` on Linux) chooses another way for one session. If the output cannot be opened the program prints a message and goes on without sound, so that songs can still be edited. If the sound crackles or is distorted, choose a larger *audio buffer* in the configuration (section 11.1). The `RMT_AUDIO_BUFFER_MS` environment variable (the size of the buffer in milliseconds, up to 50) sets another size for one session without changing the settings. The output keeps about three buffers of sound ready; `RMT_AUDIO_LATENCY_MS` (in milliseconds, up to 200) sets another amount for one session, at the cost of a later sound after a key. `RMT_AUDIO_DEBUG=1` lists the ways of output PortAudio found (the host APIs), prints the one chosen and the latency it opened, and counts the underruns.

### 2.6.1 When the sound ignores the desktop

If the sound of RITMO keeps coming out of the speakers when headphones are connected, does not change with the volume or the mute of the desktop, or RITMO is missing from the list of programs in the mixer, its sound does not go through the sound server but straight to a sound card. To find out where it goes, start RITMO from a terminal with the diagnostics on and play a song:

```
RMT_AUDIO_DEBUG=1 ./Ritmo-2.6-x86_64.AppImage 2> ritmo-audio.log
grep 'DBG audio: host API' ritmo-audio.log
```

The last of those lines names the way the sound took. `host API ALSA, device default` after the line `the default device of ALSA goes to the sound server`, and `host API PulseAudio` (the AppImage when ALSA does not go there), are the sound server. `host API ALSA, device default` without that line is the `default` device of ALSA going to a sound card; `aplay -L` confirms it: the description under `default` names the sound server (PipeWire or PulseAudio) when it goes there, and a sound card when it does not. The usual reasons, and what to do:

- The ALSA plugin of the sound server is not installed (it can also be lost in an upgrade that did not finish). With PipeWire, on Debian and Ubuntu:

  ```
  dpkg -l pipewire-alsa           # "ii" at the start: installed
  sudo apt install pipewire-alsa
  ```

  With PulseAudio the plugin is in `libasound2-plugins`. Other distributions have packages with similar names (`pipewire-alsa` on Fedora and Arch Linux).

- An old `~/.asoundrc` or `/etc/asound.conf` sends the `default` device to a sound card: rename it (for example to `.asoundrc.old`) and start RITMO again.

- With the AppImage, `RMT_AUDIO_HOST_API=PulseAudio` makes it use the sound server even when it would not choose it by itself; the line of the log says whether it found one.

After the change, `wpctl status` (PipeWire) or `pactl list short sink-inputs` lists RITMO among the programs that play, and the sound follows the output and the volume of the desktop.

## 2.7 Where the settings are kept

The configuration and the tuning belong to the user, not to the program folder, so that an update or a new installation keeps them:

| | |
|---|---|
| Linux | `~/.config/ritmo-atari.org/ritmo.conf` (`$XDG_CONFIG_HOME`) |
| Windows | the registry, `HKEY_CURRENT_USER\Software\ritmo-atari.org\ritmo` |
| macOS | `~/Library/Preferences/org.ritmo-atari.ritmo.plist` |

At the first start, if there are no settings of RITMO yet, the settings of RMT are taken over: those saved by it in the same places with the name `rmt`, or an `rmt.ini` and a `tuning.ini` next to the program.

CHAPTER 3

# Concepts

The words of the tracker have a precise meaning in this manual. Outside of it they are often used interchangeably.

**Song** A piece of music: a sequence of *song lines*. RITMO edits one song at a time; a song can start from different song lines, the *subsongs*.

**Module**
The data structure with the song, its tracks and its instruments, as it is stored in an `.rmt` file (the *module file*).

**Track**
A logical voice with the notes of a stretch of music, for example eight bars of the bass. It has a number (`$00` to `$FF`), a length (up to 256 lines) and can be used many times in a song.

**Song line**
A row of the song: it says which track plays on each channel at that moment, or where to *go to*, to loop.

**Channel**
A physical output to create sound. A mono song has four channels, `L1` to `L4`, one for each POKEY voice; a stereo song has eight, `L1`–`L4` on the left and `R1`–`R4` on the right (two POKEYs).

**Instrument**
What makes the sound at the different pitches: a volume envelope, a table of notes or frequencies, a choice of distortion, and effects such as vibrato. An instrument is a number (`$00` to `$3F`) and a name.

**Pattern**
The notes of a track with their attributes (instrument, volume, speed).

**Tracker driver**
The part of the program that makes the actual sound from the tracker data: a piece of 6502 code that runs on the emulated Atari. Different *versions* of the driver interpret the data in a different way (chapter 15).

**Mono / stereo**
Mono: the four channels of one POKEY. Stereo: two POKEYs, one for the left and one for the right output.

## 3.1 Time

The Atari plays the music once for each video frame: 50 times a second on a PAL system and 60 times on NTSC (the `NTSC` option, section 11.1). A *line* of a track lasts `S` frames, where `S` is the *speed* of the song (the `MUSIC SPEED` field). The *engine speed* says how many times the player

routine is called each frame, for the songs that need a finer time. Speeds can change in the middle of a song with the speed column of the tracks.

## 3.2 The sound of the POKEY

- Each channel has a frequency divider (`AUDF`), a volume from 0 to 15 and a *distortion* (the `AUDC` register): pure tone, noise of different kinds, buzzy tones from the polynomial counters.
- The `AUDCTL` register changes how all the channels work: it chooses the base clock (64 kHz or 15 kHz), joins two channels into a 16-bit divider, adds high pass filters, or clocks a channel with 1.79 MHz. The instruments set these bits (chapter 7).
- The notes of the tracker go from `C-1` to `C-6`: `$00` to `$3D`.

CHAPTER 4

# The main window

RITMO has one window. Under the menu bar and the toolbars the screen of the tracker is drawn: a fixed picture of the song, with its own font and its own colours, the same as in RMT. The parts of the screen are shown in figure 1.1 (page 1) and described here.

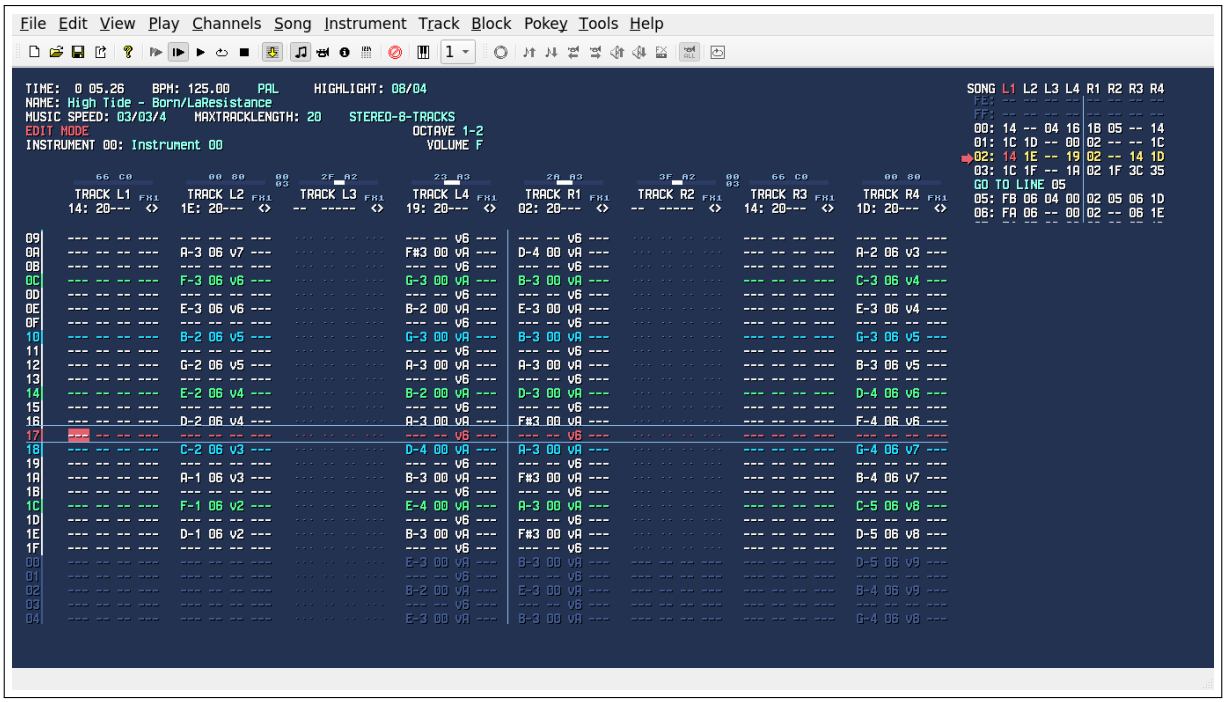



Figure 4.1. A stereo song (8 tracks) playing: the time counter runs, the song line marker follows the music.

## 4.1 The areas of the screen

**Info area (top left)**
The state of the program and the data of the song, in a few lines.

- `TIME` and `BPM`: the play time counter (when the counter is enabled in the **View** menu) and the tempo the song speed gives, in beats per minute. `PAL` / `NTSC`: the video system the speed is computed for; the number of frames per second of the screen is shown next to it.
- `HIGHLIGHT`: the two steps of the highlighted lines of the tracks (every 8 lines and every 4 lines in the figure), used to find the beat in the pattern.
- `NAME`: the name of the song and of its author, up to 64 characters.
- `MUSIC SPEED: AA/MM/S`: the current speed, the main speed and the engine speed (chapter 8). `MAXTRACKLENGTH`: the maximal length of the tracks of the song. On the same line the kind of song: `MONO-4-TRACKS` or `STEREO-8-TRACKS`.

- `EDIT MODE` (or `PROVE MODE`, and the other modes of the keyboard), the octave of the new notes (`OCTAVE 1-2`), the active instrument with its name and the volume of the new notes.

**Song area (top right)**
The sequence of song lines, one row for each line, with a column for each channel (`L1…L4`, and `R1…R4` for a stereo song). Each cell is the number of the track that plays there. The arrow marks the current song line. A line with `GO TO LINE` sends the song to another line: that is how a song loops.

**Pokey registers**
With **Pokey** → **Pokey Chip Registers** on, the registers of the POKEY (`$D200…`), the pitch, volume and distortion of each channel, as the emulated chip has them at that moment. They are drawn on the right of the song area, so they need a window about 1250 pixels wide for a mono song (they fit the 1366×768 window) and about 1720 for a stereo one, whose tracks take more room; on a narrower window they are not shown.

**Tracks area (centre)**
The tracks of the current song line side by side, one for each channel, with the lines of the pattern scrolling vertically. The header of a track names its channel (`TRACK L1`) and shows the track number, then its length and the line it loops to (`00: 20-00` is track `$00`, `$20` lines long, looping from line `$00`). `EMPTY` marks a track with no notes. The cursor is the highlighted cell.

**Status bar**
The keyboard modes (`CAP` for the caps lock) and, when it is enabled, the debug display.

A line of a track has four columns, drawn like

```
C-4 00 vA F04
```

the note, the instrument (hexadecimal), the volume (`v` and a digit) and the speed command (`F` and the speed). Empty columns are drawn `---`, `--`.

## 4.2 The four editing parts

The keyboard acts on one *part* of the screen at a time. The part is chosen with the function keys, and the cursor of the part is the red cell.

| Part | Key | What it edits |
|---|---|---|
| Tracks (`TRACK EDIT`) | `F2` | the notes of the tracks |
| Instruments (`INSTRUMENT EDIT`) | `F3` | the instrument: envelope, table, effects |
| Song (`SONG EDIT`) | `F4` | the sequence of tracks of the song |
| Info (`INFO EDIT`) | `SHIFT+F4` | the name, the speeds, the kind of song |

`ENTER` returns from the song or info part to the tracks or instruments part you came from.

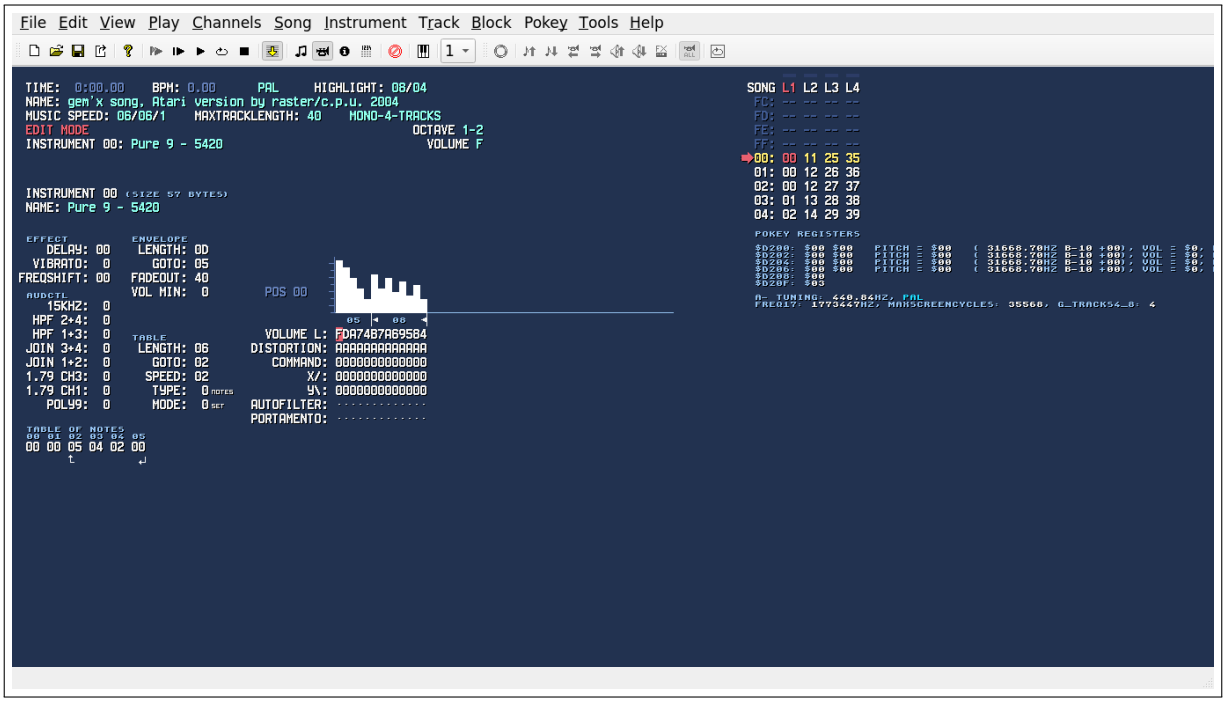



**Figure 4.2.** The instrument part (`INSTRUMENT EDIT`) with an instrument loaded.




**Figure 4.3.** The song part (the top right of the window): the cursor is in the song area, on the track of channel L1 in song line 00.




**Figure 4.4.** The info part (the top left of the window): the cursor is on the name of the song, which is drawn in red.

## 4.3 Edit mode and prove mode

The window is in one of two modes, shown in the info area. In `EDIT MODE` the keys write notes. In `PROVE MODE` (also called *jam* mode) editing is switched off and the note keys only play: you can try an instrument or play along with the song without changing it. `CONTROL+SPACE` (or **Edit** → **Switch Edit Mode**) toggles between them. The instrument and info parts behave as in edit mode; hold `SHIFT` to play notes while there.

## 4.4 The menus and the toolbars

The menu bar has the menus **File**, **Edit**, **View**, **Play**, **Channels**, **Song**, **Instrument**, **Track**, **Block**, **Pokey**, **Tools** and **Help** (chapter 10). Under them there are two toolbars: the *main toolbar* (new, open, save, export, the play buttons, the edit modes, MIDI on/off…) and the *block toolbar* (transpose, volume, instrument change, play the block…). Both can be shown or hidden in the **View** menu, and the choice is kept in the configuration; the icons follow the interface size. The buttons have a tip with the name of the command and its key.

## 4.5 The mouse

The program is made for the keyboard, but the mouse helps.

- In the track area: the *left* button on a track header switches the channel on or off, the *right* button plays the channel alone (solo), a second click puts all the channels back.
- In the volume envelope of the instrument: the left button draws the volume, the right one sets it to zero.
- In the info area, on the fields `MAXTRACKLENGTH`, `MONO/STEREO` and `PAL/NTSC`: a click toggles the field or opens the dialog that changes it, then applies the change to the song.
- The wheel over `VOLUME` or `OCTAVE` of the info area changes the volume or the octave of the new notes.

CHAPTER 5

# Your first song

This chapter makes a short song from nothing, to see how the parts of the program fit together. Every key used here is explained in the following chapters.

## 5.1 A new song

Press `CONTROL+N` (**File** → **New**). The dialog of figure 5.1 asks for the *maximal length of the tracks* (how many lines a track can have; 64 is a good start) and for the kind of song: `MONO - 4 TRACKS` or `STEREO - 8 TRACKS`. Choose mono. The current song is discarded, so the dialog warns you.

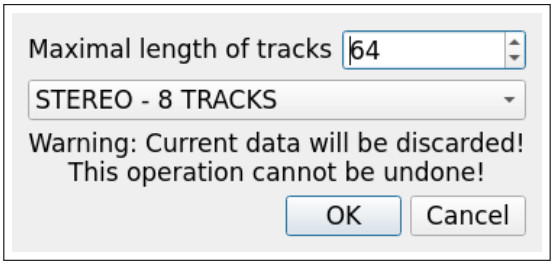



**Figure 5.1.** The **New** dialog.

## 5.2 Entering notes

The cursor is in the first column of the first track, the note column. The keyboard is a piano (figure 5.2): the lower row of letters, `ZXC`…, plays from `C-1` and the upper row, `QWE`…, from `C-2`; the keys of the row above each are the sharps (`S`, `D`…and `2`, `3`…). The exact keys depend on the keyboard layout chosen in the options (QWERTY, QWERTZ or AZERTY): the tables of appendix A show every key.

```
1   2   3   4   5   6   7   8   9   0   -   =
   C#2 D#2     F#2 G#2 A#2     C#3 D#3     F#3
 Q   W   E   R   T   Y   U   I   O   P   [   ]
C-2 D-2 E-2 F-2 G-2 A-2 B-2 C-3 D-3 E-3 F-3 G-3
  A   S   D   F   G   H   J   K   L   ;   '
     C#1 D#1     F#1 G#1 A#1     C#2 D#2
   Z   X   C   V   B   N   M   ,   .   /
  C-1 D-1 E-1 F-1 G-1 A-1 B-1 C-2 D-2 E-2
```

**Figure 5.2.** The note keys of the QWERTY layout.

Press `Z`: the note `C-1` is written with the active instrument and the volume of the info area, it is played, and the cursor moves down by the number of lines of the *note spacing* (the combo of the toolbar, default 1). Press `X`, `C`…to write a scale. `SPACE` deletes what is under the cursor, `BACKSPACE` deletes the note.

**Tip**

`numblock /` and `numblock *` lower and raise the octave of the new notes, `numblock +` and `numblock -` change the spacing of the notes. The volume of the new notes is changed with `CONTROL+numblock +` and `CONTROL+numblock -`.

## 5.3 Choosing the instrument

The instrument is the sound of the note. The active instrument is shown in the info area (`INSTRUMENT 00: …`). `SHIFT+LEFT` and `SHIFT+RIGHT` change it. To see what an instrument is, press `F3`: the instrument part of figure 4.2 appears. Chapter 7 explains it; the quickest way to get good sounds is to load instruments: **Instrument** → **Load instrument from file…** and choose one of the `.rti` files in the `instruments` folder of the songs and instruments download (`Ritmo-Songs-and-Instruments-`*version*`.zip` of the release, chapter 2; the program itself does not carry them).

## 5.4 Playing

- `F5` plays the song from the start, `F7` from the cursor, `F6` loops the current song line and `ESC` stops.
- `ENTER` plays the note under the cursor.
- `F12` makes the screen follow the playing position.

## 5.5 More tracks and song lines

The notes of a track belong to a *song line*. To write more music, go to the song area with `F4`: each column is a channel and each number a track. Press `CONTROL+T` to put a new empty track in the current song line and channel, and `F2` to come back and write in it. `CONTROL+D` duplicates the track under the cursor, to change a copy. `CONTROL+G` makes the current song line a `GO TO LINE`, which loops the song.

## 5.6 Saving and exporting

`CONTROL+S` saves the song as an `.rmt` file. To use it on the Atari, **File** → **Export** makes a SAP file (to play with the Atari players), an executable `.xex` that plays the song when started, assembler source, or a WAV file (chapter 12).

**Note**

Test the finished song on the Atari or in an Atari emulator, when you can: RITMO plays it with an exact emulation of the player routine, but the way the notes sound depends on the *tracker driver version* (chapter 15) and on the video system, PAL or NTSC.

CHAPTER 6

# Editing tracks

The tracks are edited in the TRACK EDIT part (F2). The complete list of keys of the part is in appendix A; this chapter explains what they do.

## 6.1 The cursor

The arrow keys, TAB and SHIFT+TAB, PAGE UP and PAGE DOWN move the cursor in the tracks. The columns of each track are, from the left: the note, the instrument, the volume and the speed. The key you type goes to the column under the cursor.

| Column | What you type |
|---|---|
| Note (NNN) | a note key; numblock 1–6 change the octave of the note under the cursor; BACKSPACE deletes the note and the instrument |
| Instrument (TT) | 0–F, from $00 to $3F. If the note column of the line is empty the column behaves like a note column: you can type a note there |
| Volume (vV) | 0–F. A volume can be written alone, with no note |
| Speed (FSS) | 0–F: a new speed for the song from this line on, from $01 to $FF |

If the *respect volume* mode (F11) is on, a note typed on a line that has a volume does not replace the volume.

## 6.2 The length of a track

A track is as long as its *end line*: CONTROL+END sets or clears it. HOME and END go to the start and the end of the track; CONTROL+HOME sets the start of a *wise loop*, the line the track goes back to when it reaches its end, so that a repeated phrase is written once. The header of the track shows both numbers (chapter 4).

INSERT and DELETE (or CONTROL+I and CONTROL+U) insert and delete lines in the track.

## 6.3 Playing while editing

- ENTER plays the note at the cursor, SHIFT+ENTER plays it and makes its instrument and volume the active ones (a quick way to "pick up" a sound from a song), and CONTROL+ENTER plays all the notes of the line.
- SHIFT+tonekey plays a note with the active instrument and volume on the current channel, without writing it, and SHIFT+CONTROL+tonekey plays it on both stereo channels.
- F9 switches the current channel off and on, CONTROL+F9 plays it alone, CONTROL+1–8 toggle the channels one by one.

## 6.4 Blocks

A *block* is a selection of lines in a track.

1. Select with SHIFT+UP, SHIFT+DOWN, SHIFT+HOME and SHIFT+END; CONTROL+A selects all the data of the track. ESC deselects.
2. Copy (CONTROL+C), cut (CONTROL+X), paste (CONTROL+V), merge with the content under the cursor (CONTROL+M) and exchange the block with the clipboard (CONTROL+E). With no block the data at the cursor is taken.
3. CONTROL+B restores the block from a backup that is made as soon as the block is started: the way back from a wrong modification.

The blocks can also be changed with keys, either on every line of the block, or only on the lines with the active instrument (SHIFT+CONTROL+A chooses): CONTROL+F1/F2 transpose by semitones and CONTROL+F3/F4 by octaves, SHIFT+CONTROL+LEFT/RIGHT change the instrument and SHIFT+CONTROL+UP/DOWN the volume. The block toolbar has the same buttons.

### 6.4.1 The Effects and Tools dialog

CONTROL+F (**Block** → **Effects/Tools...**) opens the window of figure 6.1: a choice of effects for all the data of the block, with the fields of the chosen effect. The effects include fading the volume in or out between two levels in percent; **Try** applies the effect, **Restore** takes it back, **Play/Stop** plays the block while you are adjusting it and **Default** sets the fields to their default values.

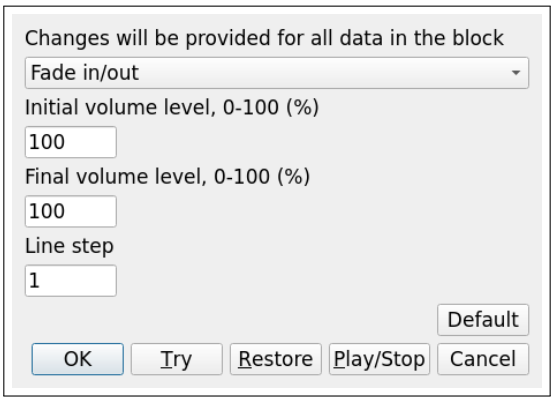



**Figure 6.1.** The block effects (CONTROL+F).

## 6.5 Tracks as objects

The **Track** menu works on a whole track: copy, paste, cut, delete, load a track from a file or save it as a text file, ask for information about the current track, *search and build a wise loop* in a track (the program finds the repeats and writes them once) or *expand* it again, and the clean-up operations that remove duplicated or unused tracks from the song. The operations on all the tracks change the whole song; save it before using them.

## 6.6 Undo

CONTROL+Z undoes the last change and CONTROL+Y does it again. **Edit** → **Clear Undo & Redo History** forgets the changes made so far.

CHAPTER 7

# Instruments

An instrument says how a note sounds from the moment it starts: how its volume changes with time, which distortion the POKEY uses, whether the pitch moves. In `INSTRUMENT EDIT` (`F3`) the instrument with the number in the info area is shown and edited (figure 4.2); `SHIFT+LEFT` and `SHIFT+RIGHT` choose another one, from `$00` to `$3F`.

## 7.1 The parts of an instrument

**`NAME`** The name, up to 32 characters.

**Envelope**
A sequence of steps, one for each frame, from 1 to 32. Each step has the volume (`VOLUME L`, and `VOLUME R` for the right channels of a stereo song), the `DISTORTION` (the POKEY distortion of the step, an even value from 0 to E), a `COMMAND` with its parameters `X` and `Y`, and the flags `AUTOFILTER` and `PORTAMENTO`.

- `ENVELOPE LENGTH`: the number of steps; `ENVELOPE GOTO`: the step to jump to when the end is reached.
- `FADEOUT`: a volume slide that starts when the end of the envelope is reached for the first time (`$00` no slide, `$FF` the fastest), down to the volume `VOL MIN`.

**Table of notes / frequencies**
Up to 32 values added to the note or to the frequency, one for each `TABLE SPEED` frames: arpeggios, pitch bends, drums. `TABLE TYPE` chooses between notes (in semitones) and frequencies; `TABLE MODE` between adding to the base note (`SET`) and adding to the last result (`ADD`). `TABLE LENGTH` and `TABLE GOTO` work as in the envelope.

**Effects**
`DELAY` (the number of frames before the effects start), `VIBRATO` (three levels) and `FREQSHIFT` (a frequency shift for each frame).

**`AUDCTL`**
The bits of the POKEY control register the instrument sets: the 15 kHz base clock, the two high pass filters, the joined 16-bit channels, the 1.79 MHz clocks and the 9-bit polynomial counter.

The fields, their ranges and the meaning of the envelope commands and of the distortions are in the reference tables of appendix A.

## 7.2 Editing

The cursor moves with the arrow keys and `TAB`, which jumps between the groups of parameters. The values are typed in hexadecimal (`0`–`F`) or changed with `CONTROL+arrow`. In the tables and the envelope, `INSERT` adds a step, `SHIFT+INSERT` duplicates one, `DELETE` removes one and `SPACE` clears one;

`HOME` and `END` go to the start and to the end, and `CONTROL+HOME` and `CONTROL+END` set the loop and the length.

The volume envelope can be drawn with the mouse: the left button sets the volume of a step, the right one sets it to zero. `SHIFT+CONTROL+UP` and `DOWN` move all the steps at once; `SHIFT+CONTROL+numblock +` and `-` change the volumes of the left and right envelopes.

**Tip**

Press `SHIFT+ENTER` on a note of a track to pick up its instrument and listen to it, and `SHIFT` plus a note key while the instrument is on the screen to hear the changes as you make them.

## 7.3 Instrument files and the Instrument menu

The **Instrument** menu has the commands to copy, paste, cut and delete the instrument, to paste only a part of it (only the volume envelopes, only the parameters, only the table…), to show information about the instrument (where it is used), to change all its occurrences in the songs to another number, to renumber all the instruments, to clear the unused ones, to *load* an instrument from a file and to *save* it as an `.rti` file. The `instruments` folder of the songs and instruments download (`Ritmo-Songs-and-Instruments-`*version*`.zip` of each release) has files to start from.

## 7.4 The distortions

The `DISTORTION` of an envelope step makes the character of the sound. The values are `0` and `8` (white noise), `2` (square-ish tones), `4` (pure by default, and a table of its own in the patched drivers), `6` and `C` (buzzy bass tones), `A` (pure tones) and `E` (gritty bass tones). The exact POKEY registers behind each are in the table of appendix A, and some of the tracker drivers (chapter 15) give some of them a different sound.

## 7.5 The instrument active help

**View** → **Instrument Active Help** shows, below the instrument, a description of the value under the cursor in the envelope: what the distortion or the command is, and the effect of its parameters.

CHAPTER 8

# The song

## 8.1 Song lines

The song area (the SONG EDIT part, F4) lists the song lines. Each column is a channel, and the number in a cell is the track the channel plays on that line. The keys:

- 0–F, CONTROL+LEFT, CONTROL+RIGHT and BACKSPACE change the track number of the cell.
- CONTROL+T puts a new empty track in the cell, CONTROL+D makes a copy of the track and puts it in its place (the program asks when the track was used only once), CONTROL+J prepares a whole line of empty tracks.
- INSERT, DELETE (CONTROL+I, CONTROL+U) insert and delete song lines. The GO TO LINE values that point past the change are adjusted by the program.
- CONTROL+K inserts a copy or a clone of song lines, in the dialog of figure 8.1.
- CONTROL+G makes the line a GO TO LINE: the song continues from another song line, which loops it.
- HOME and END, SHIFT+PAGE UP and SHIFT+PAGE DOWN move to the start or the end of the song and between the subsongs.

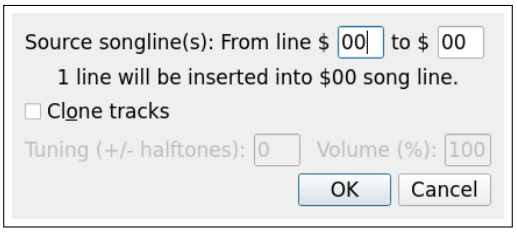



Figure 8.1. Insert a copy or a clone of song lines.

## 8.2 Subsongs

A song may contain more than one tune: each subsong starts from a song line. The keys SHIFT+PAGE UP and SHIFT+PAGE DOWN move between them, and the SAP export (chapter 12) lists them in its *Subsongs* field: the hexadecimal numbers of the song lines that start the subsongs, the first of which is the default one.

## 8.3 Bookmarks

F8 sets a bookmark at the cursor and CONTROL+F8 clears it. SHIFT+F7 plays from the bookmark, a way to listen to the same passage again and again while you work on it.

## 8.4 The info area

The `INFO EDIT` part (`SHIFT+F4`) edits the data of the song itself.

| Field | Meaning |
|---|---|
| `NAME` | the name of the song and of the author, up to 64 characters |
| `MUSIC SPEED: AA/MM/S` | `AA` is the current speed of the song, `MM` the main speed, which the song uses when it is played from the beginning (both from `$01` to `$FF`); `S` is the engine speed, from 1 to 8: how many times the player routine is called in each frame. A speed above 4 is not supported by the exported formats |
| `MAXTRACKLENGTH` | the maximal length of the tracks |
| `MONO / STEREO` | the number of tracks: 4 or 8 |
| `PAL / NTSC` | the video system, 50 or 60 frames per second |

The values are changed with the hexadecimal keys, or `CONTROL+arrow`. `TAB` and `LEFT`/`RIGHT` move between the fields, `SHIFT+TAB` goes to the name and `ENTER` returns to the part you came from.

## 8.5 Changing the kind of song

The commands of the **Song** menu that change the whole song open dialogs of their own.

- **Switch song between 4 or 8 channels...** changes a mono song to stereo and the other way round. The data of the channels that disappear is lost, and the dialog says so.
- **Song columns' order change/copy/clear...** (figure 8.2) reorders the columns of the song, copies one into another, turns a mono column into a stereo one or clears a column, for the song lines in a range.
- **Change maximal length of tracks...** (figure 8.3) changes the maximal length of the tracks.
- **All size optimizations...** runs the optimizations that make the song smaller: searching wise loops in all the tracks, clearing the unused ones, ...(section 8.6).

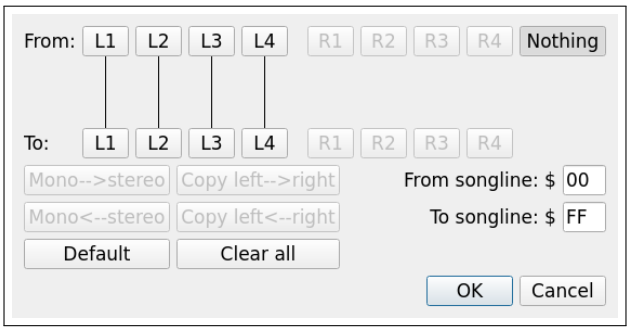



Figure 8.2. The order of the columns of the song.

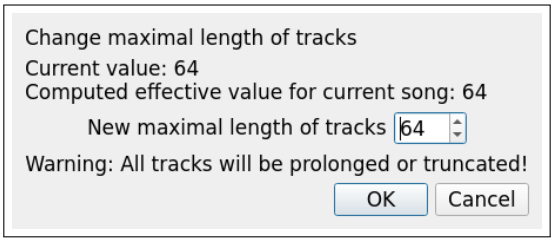



**Figure 8.3.** The maximal length of the tracks.

## 8.6 Optimizing the size

The Atari has little memory, so a song should be small. The program has the tools to make it so: the *wise loops* (a track that repeats a phrase is stored once), the clearing of the tracks and the instruments that are not used in the song, the removal of duplicated tracks (the song is adjusted to the one left), and the *cleanup* commands of the **Track** and **Instrument** menus. The export for programmers (*RMT stripped*) shows the features used by the song, which lets a player routine be made smaller still (chapter 12).

CHAPTER 9

# Playing and monitoring

## 9.1 Play commands

| Key | Action |
|---|---|
| F5 | play the song from the start |
| F7 | play from the cursor |
| SHIFT+F7 | play from the bookmark |
| F6 | play the current song line in a loop |
| SHIFT+F6 | play the selected block in a loop |
| ESC | stop and silence |
| F12 | follow mode: the screen follows the playing position |
| CONTROL+F12 | toggle PAL / NTSC |
| F11 | respect volume on/off |

The same commands are the buttons of the main toolbar and the entries of the **Play** menu. With the option *Reset the Atari sound routines each time ESC is pressed* (on by default) ESC also starts the player routine again, which clears sounds that hang. A song with the AUTOFILTER that is opened while the Patch16 driver is selected makes the program ask whether to play it with the original RMT 1.28 driver (chapter 15).

## 9.2 Channels

The **Channels** menu and the keys switch the channels on and off to hear a part alone: F9 mutes or unmutes the active channel, SHIFT+F9 all the channels, CONTROL+F9 plays the active channel alone (solo) and again unmutes everything, CONTROL+1–8 toggle a channel. The mouse does the same on the track headers (chapter 4).

## 9.3 Watching the sound

- The *play time counter* (**View** → **Play Time Counter**) shows the time and the tempo while the song plays.
- The *volume analyzer* (**View** → **Volume Analyzer**) draws the volume of each channel above the track headers; figure 9.1 shows it.
- **Pokey** → **Pokey Chip Registers** shows the registers of the POKEY with the pitch, volume and distortion of each channel: the real thing the Atari would do.

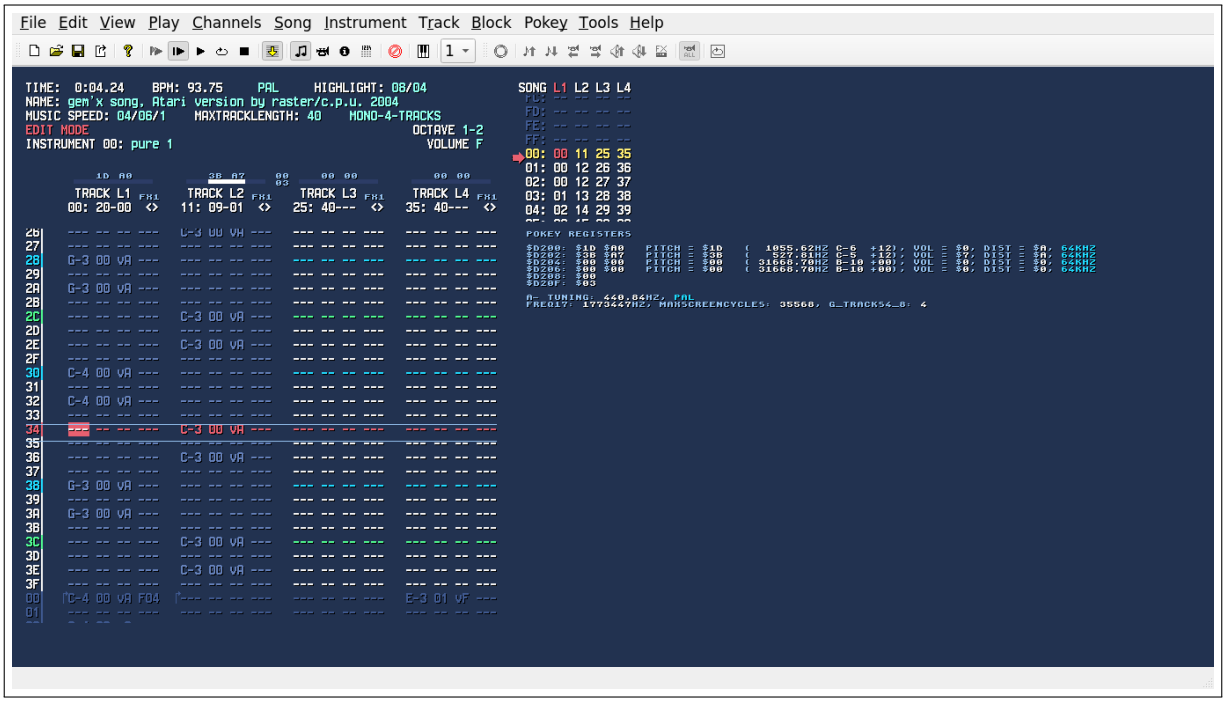



**Figure 9.1.** The volume analyzer and the registers of the POKEY while a song plays.

## 9.4 Playing from the keyboard

In prove mode (CONTROL+SPACE) the note keys only play. In edit mode SHIFT plus a note key plays without writing. A MIDI keyboard does the same and can also record (chapter 13).

CHAPTER 10

# The menus

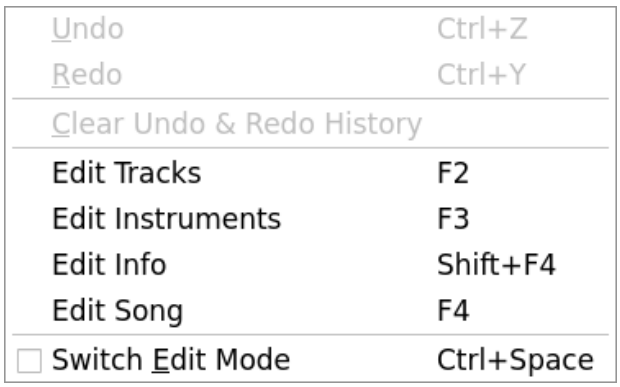
This chapter goes through the menu bar. The table of every command with its key and its short description is in appendix A (it is written by the program, so it is always up to date); here each menu is shown and the commands that need an explanation are explained.

## 10.1 File



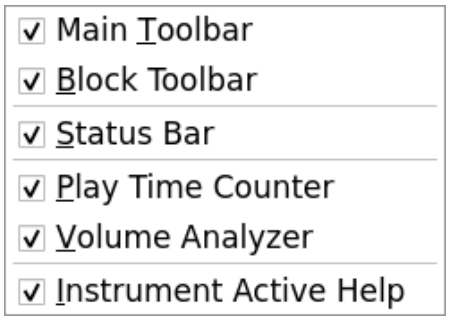

- **New**, **Open...**, **Reopen** (loads the song from the last saved version), **Save**, **Save As...**: the song is an `.rmt` file; `.txt` and `.rmw` are chosen in the file type of the dialog.
- **Import...**: ProTracker modules and TMC songs (chapter 12).
- **Export**: the formats for the Atari (chapter 12).
- **Print**, **Print Preview**, **Print Setup**: the screen of the song is printed on a landscape page, scaled to fill it.
- **Properties...** (`ALT+ENTER`): the module properties: what the module holds, its video standard, tracker driver and tuning (section 11.2).

## 10.2 Edit



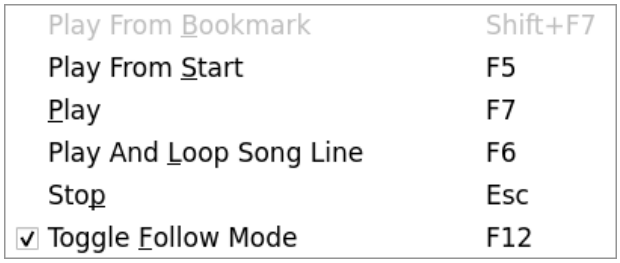

Undo and redo (the history can be cleared), the four parts of the screen (tracks, instruments, info, song), and the switch between edit and prove mode.

## 10.3 View




Shows or hides the toolbars and the status bar, the play time counter, the volume analyzer and the instrument active help. The choices are kept in the configuration.

## 10.4 Play



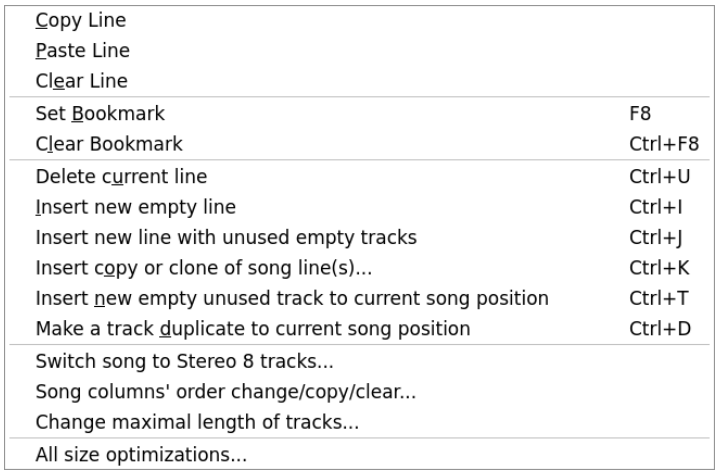

Play from the bookmark, from the start, from the cursor, loop the song line, stop and follow mode (chapter 9).

## 10.5 Channels



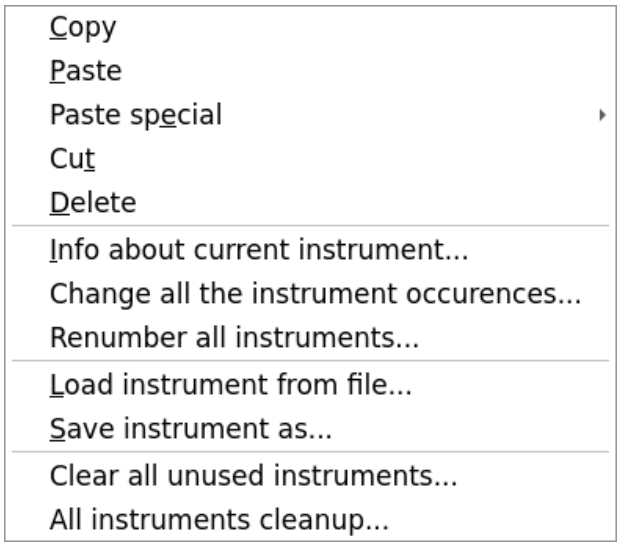

Switch on and off each of the channels 1 to 8, mute or unmute the active one or all of them, or play the active one alone.

## 10.6 Song




Copy, paste and clear a song line; the bookmark; insert and delete lines; insert tracks; and the commands that change the whole song (chapter 8).

## 10.7 Instrument



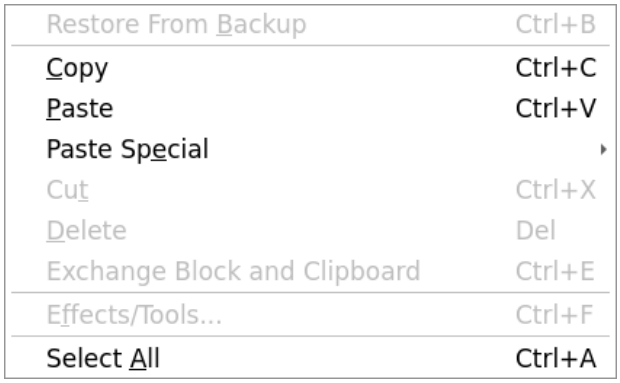

Everything to do with an instrument: copy and paste (and paste special), information, change or renumber, clear the unused ones, load and save (chapter 7).

## 10.8 Track



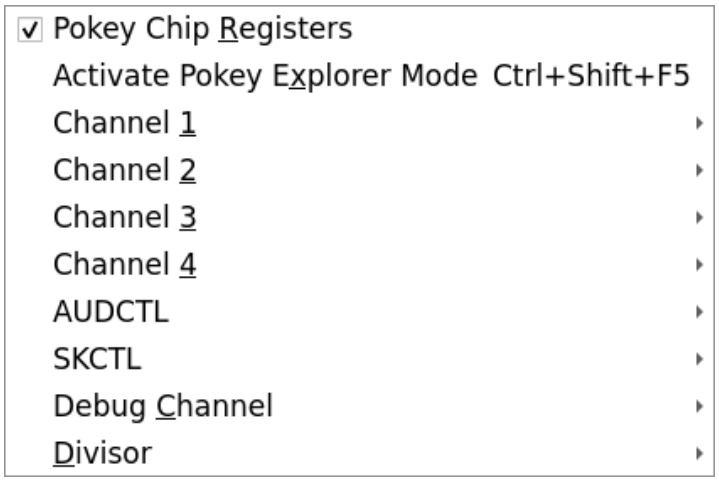

Copy, paste, cut and delete a track; the step size; information about the current track; wise loops; renumbering; loading and saving a track; and the clean-up of the tracks of the song.

## 10.9 Block



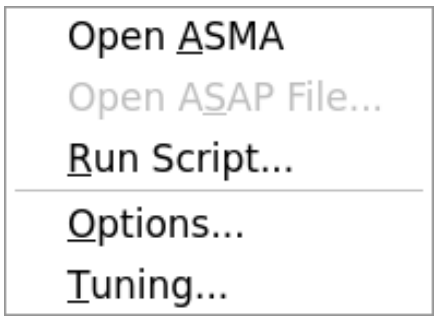

The commands of the blocks (chapter 6): restore from the backup, copy, paste, paste special (merge, only the volumes, only the speeds), cut, delete, exchange, effects and tools, select all.

## 10.10 Pokey




The registers of the chip and the Pokey explorer mode, with a submenu for each channel (chapter 14).

## 10.11 Tools




**Open ASMA** opens the Atari SAP Music Archive in the browser; **Run Script...** runs a script (chapter 16); **Options...** and **Tuning...** open the dialogs of chapter 11.

## 10.12 Help




**Help** (F1) opens this manual, and **Online Help** (SHIFT+F1) the reference of the original RMT (menus, keys, instruments), both in the browser, from the GitHub page of RITMO (they need the Internet). **About RITMO** shows the version and the credits; on a small screen, or with large fonts, its text scrolls.

CHAPTER 11

# Options and tuning

## 11.1 The Options dialog

**Tools** → **Options...** opens the dialog of figure 11.1. The settings are saved when the program closes, in the settings of the system (chapter 2).

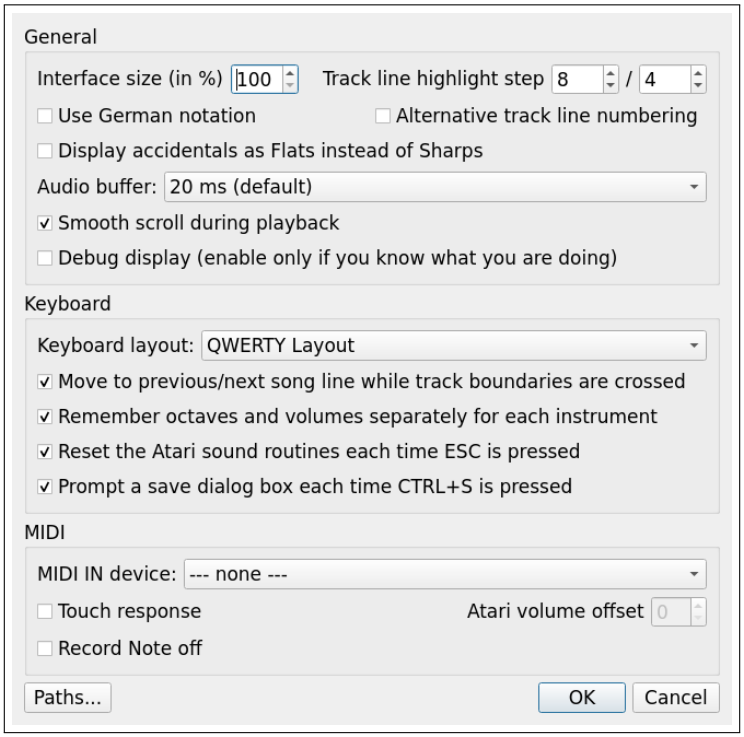



**Figure 11.1.** The **Options** dialog.

### 11.1.1 General

**Interface size (in %)**
The size of the drawing of the window, from 100% to 300%. Only whole multiples of 100% keep the pixels exact. The first start chooses it from the size of the screen (section 2.5).

**Track line highlight step**
The two numbers of the `HIGHLIGHT` field of the info area: every how many lines the lines of the tracks are highlighted (for example 8 and 4).

**Use German notation**
The note `B` is written `H`, as in the German notation.

**Alternative track line numbering**
The line numbers follow the highlight step.

**Display accidentals as flats instead of sharps**
`D#` is written as `Eb`.

**Audio buffer**
How much sound the output of the system takes at a time, from 5 to 40 milliseconds. A small buffer makes the sound follow the keys and the MIDI keyboard sooner; a larger one keeps the sound clean when the computer or the sound server is busy. The default is 20 ms on Linux, where the sound servers (PipeWire, PulseAudio) work in pieces of about that size, and 5 ms on Windows and macOS. The output keeps about three buffers of sound ready (60 ms with the 20 ms buffer), so that a short stall of the sound server is not heard as a gap. If the sound crackles or is distorted, choose 30 or 40 ms.

**Smooth scroll during playback**
The lines of the tracks scroll one pixel at a time while a song plays.

**Debug display**
A line of internal values in the status bar, off by default. Enable it only if you know what it shows.

The video standard and the tracker driver are not here: they belong to the module, and are set in its properties (section 11.2).

### 11.1.2 Keyboard

**Keyboard layout**
QWERTY, QWERTZ or AZERTY: where the piano of the note keys is. The first start picks the layout from the language of the keyboard.

**Move to previous/next song line while track boundaries are crossed**
The cursor passes from the end of a track to the next song line, and the other way round.

**Remember octaves and volumes separately for each instrument**
Each instrument keeps the octave and the volume you last used with it.

**Reset the Atari sound routines each time ESC is pressed**
`ESC` also starts the player routine again.

**Prompt a save dialog box each time CTRL+S is pressed**
`CONTROL+S` asks where to save, also from the menu and the toolbar.

### 11.1.3 MIDI

The MIDI input device, the touch response, the Atari volume offset and *Record note off*: they are described in chapter 13.

### 11.1.4 Paths

The **Paths...** button opens a dialog with the default folders of the songs, the instruments and the tracks, each with a **Browse** button. The dialogs to open and save start there.

## 11.2 The module properties

**File** → **Properties...** (ALT+ENTER) opens the dialog of figure 11.2: what the module holds (the file, the channels, the speeds, the instruments and tracks used, the size of the RMT module), then the settings that belong to the module rather than to the program. They are saved with the other settings when the program closes.

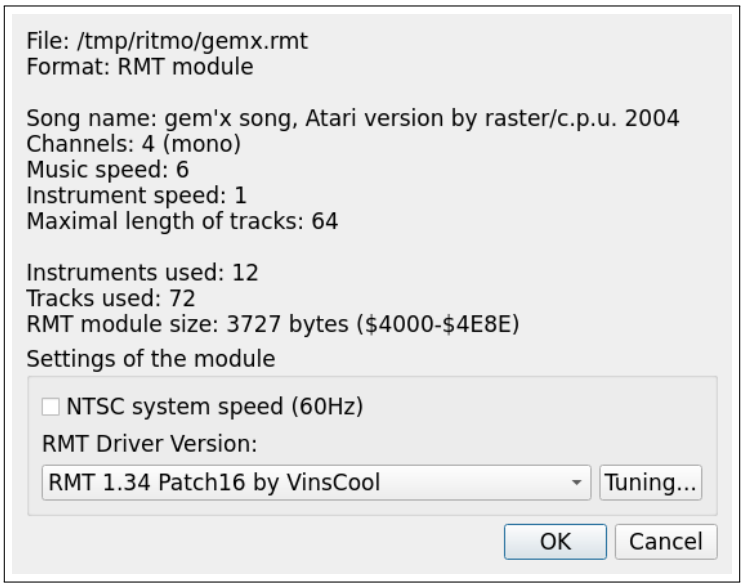



**Figure 11.2.** The **Module Properties** dialog.

**NTSC system speed (60 Hz)**
The video system of the song: 60 frames a second instead of the 50 of PAL. The same switch is CONTROL+F12.

**RMT Driver Version**
The tracker driver, the player routine that makes the sound while you work (chapter 15). The driver must be the one the song was made with; the default is *RMT 1.34 Patch16 by VinsCool*. The driver chosen here is also the one new and opened songs start with. A song that uses the Autofilter, opened with the RMT 1.28 Unpatched driver at its question, plays with that driver for itself only: the dialog says so, and the default stays as it was.

**Tuning...**
The tuning of section 11.3. The button is enabled only for *RMT 1.34 Patch16*, the only driver whose frequency tables the tuning reaches: the others play the tables inside their player.

## 11.3 The tuning

**Tools** → **Tuning...** (figure 11.3) is a calculator for the pitch of the notes. The POKEY makes its frequencies by dividing a clock, so not every pitch exists: the program computes the divisors that come nearest to the notes of the temperament you choose, and the tracker plays with them.

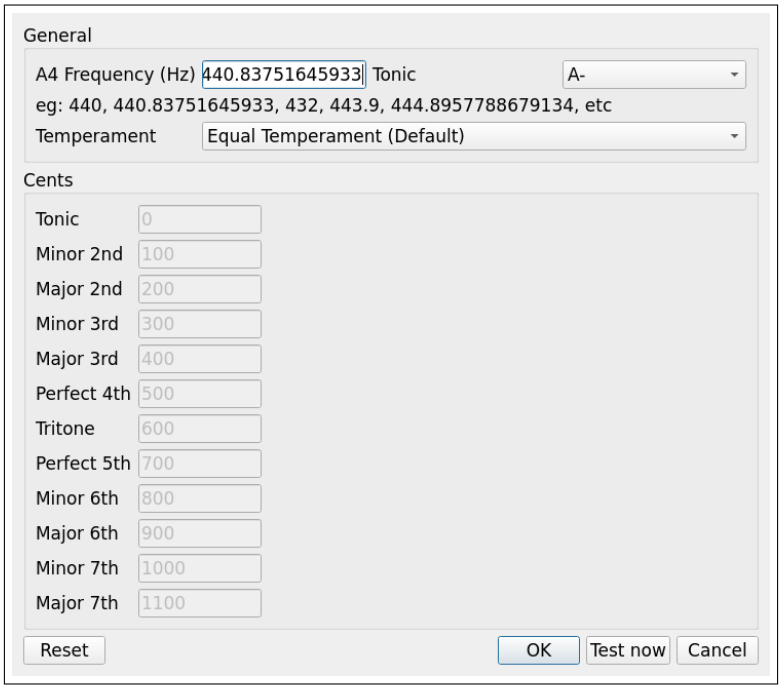



**Figure 11.3.** The **Tuning** dialog.

**A4 Frequency (Hz)**
: The pitch of the note `A-4`, in every key: by default 440.8375… Hz (PAL) or 444.8957… Hz (NTSC), the tuning RMT has always had, which the tables of the Patch16 player were made with. It can be any number between 6.875 and 7040: `440`, `432`, `443.9`…

**Tonic**
: The key the tables are tuned to, by default `A-`: the note that gets the temperament's 0 cents. The program derives its frequency from the A4 frequency through the temperament, so `A` always sounds at the A4 frequency; in equal temperament every tonic gives the same notes.

**Temperament**
: *Equal Temperament* (the default), one of 21 historical temperaments (in alphabetical order), or the *Custom Temperament in Cents*.

**Cents**
: The twelve notes of the octave as cents above the tonic (100 cents are an equal-tempered semitone, the octave is 1200). They show the temperament chosen; only those of the custom temperament can be typed, between 0 and 1200 (the tonic stays 0), and they are kept while another temperament is shown.

**Reset**
: puts back the values the dialog was opened with; **Test now** tries the tuning.

A `tuning.ini` of an earlier version is still read: its ratios become cents, and a temperament that is no longer in the list (the seven that are no 12-note temperament with a 2:1 octave) plays as equal temperament.

The tuning is saved with the configuration and is also the one a script uses when it exports a song (chapter 16).

CHAPTER 12

# Import and export

## 12.1 Importing

**File** → **Import...** reads a ProTracker module (`.mod`) or a TMC song (`.tmc`, `.tm8`) and makes a song of it. After the file is chosen the dialog of figure 12.1 asks how to convert a module.

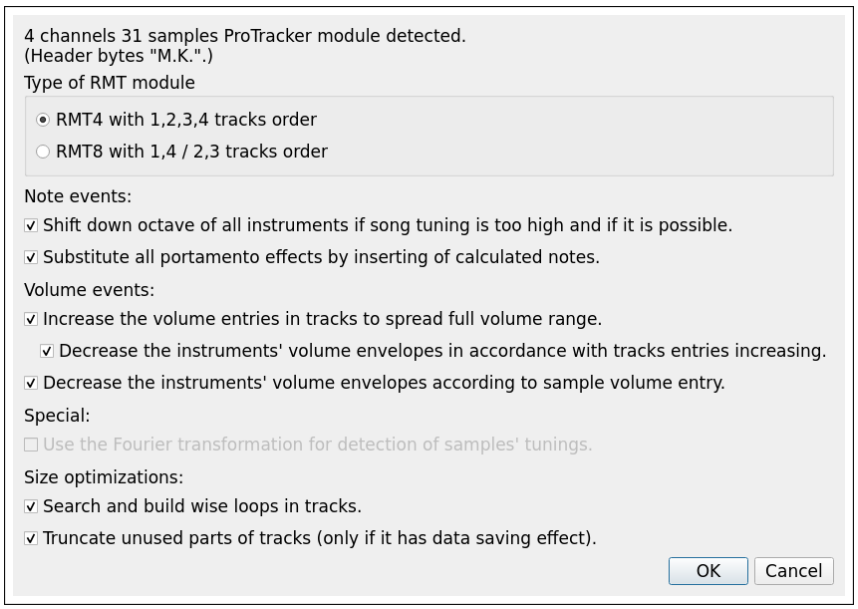



**Figure 12.1.** Importing a ProTracker module.

- The *type of RMT module*: `RMT4` (the four channels of the module on the four tracks) or `RMT8` (the channels spread on the two POKEYs, in the order 1, 4 / 2, 3).
- The treatment of the *notes* (shift the octave down if the tuning of the song is too high, replace the portamento effects by calculated notes), of the *volumes*, and the *size optimizations* that are made while importing (wise loops, cutting the unused parts of the tracks).

When the conversion is done a dialog reports how many tracks, instruments and song lines the song has, and what the optimizations did. Check the box and press **OK**: the song is loaded, but it still needs your work. The instruments are rough: look at their envelopes and distortions (drums, basses...) and correct the tunings of the original samples.

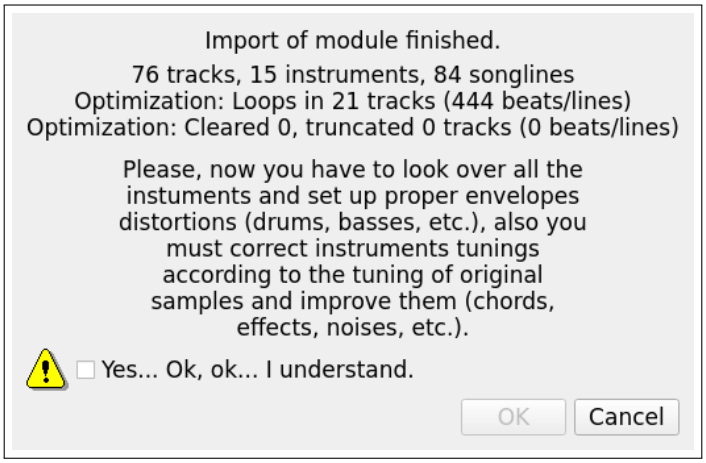



**Figure 12.2.** The result of the import.

## 12.2 Exporting

**File** → **Export** writes the song in a format for the Atari. The file dialog chooses the name and the format; most of the formats then show a dialog of options. Every format is written from the data of the song as the tracker plays it.

The formats that play the song to make the file (SAP-R, LZSS, SAP, XEX with the LZSS player, WAV) can take a few seconds, more on a slow computer, and RITMO cannot be used meanwhile: when the work takes longer than a moment, the whole window is covered with grey and a spinner turns in the middle, until the file is written (figure 12.3). It appears only when the work lasts longer than 150 milliseconds, so the quick formats and the dialogs of the options never show it, and it disappears by itself at the end: the window is then as it was.

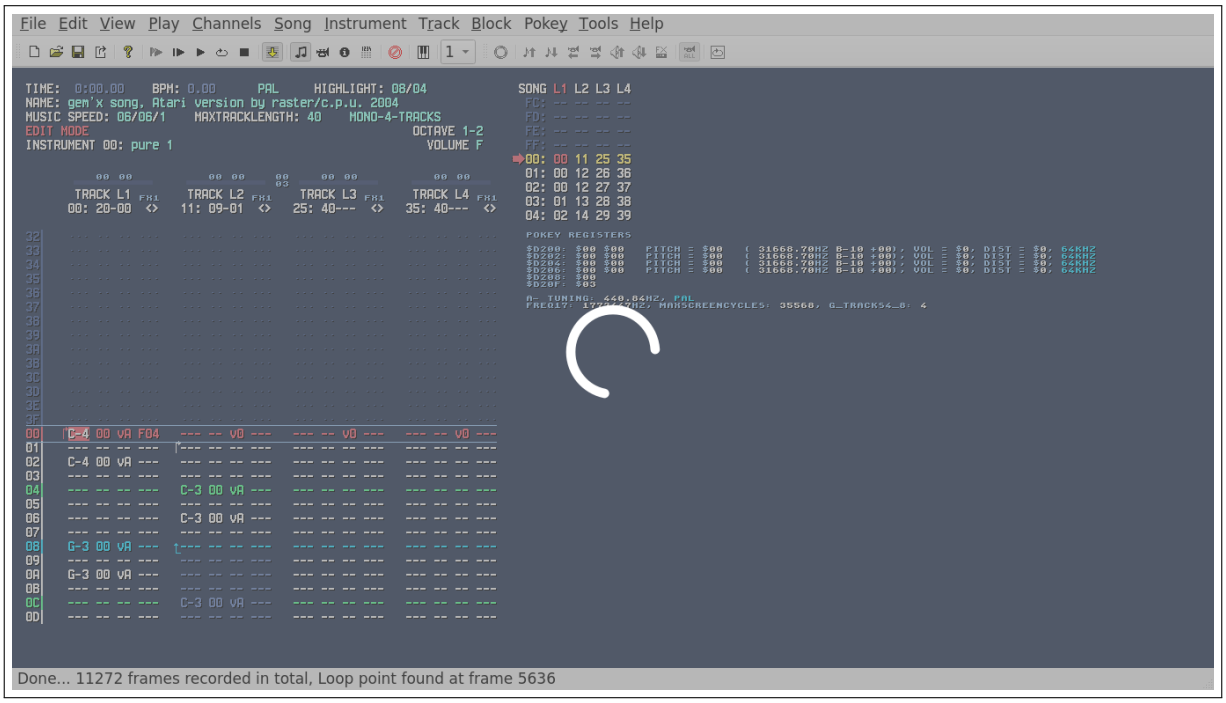



**Figure 12.3.** An export at work: the window covered with grey, the spinner turning in the middle.

| Format | Use |
|---|---|
| RMT stripped song file (`.rmt`) | the module without the names and the unused tracks and instruments, to link in a program that uses the RMT player routine |
| ASM simple notation (`.asm`) | the notes as assembler data, for your own player |
| SAP-R data stream (`.sapr`) | the register writes of the POKEY, frame by frame |
| Compressed SAP-R (`.lzss`) | the same, compressed, for the LZSS player |
| SAP file + LZSS driver (`.sap`) | a file the Atari players (ASAP, the plug-ins of many players, ASMA) play directly |
| XEX Atari executable (`.xex`) | a program for the Atari that plays the song when it is started |
| XEX Atari executable + RMT player (`.xex`) | the same with the RMT player and the module of the song |
| Relocatable ASM for RMTPlayer (`.asm`) | the module as source with labels, to be placed in memory by your program |
| WAV audio file (`.wav`) | the sound as a 16-bit, 44.1 kHz, stereo file, the song played once up to its loop |

### 12.2.1 RMT stripped

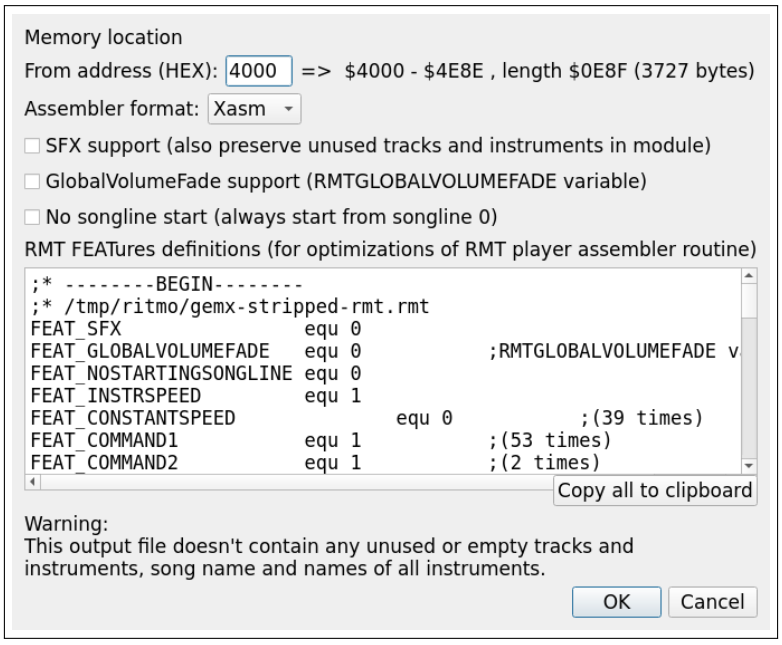



**Figure 12.4.** The export of an RMT stripped file.

The *from address* is where the module will be in the memory of the Atari; the program shows the range it takes. The options are *SFX support* (keep the unused tracks and instruments, for sound effects played by the game), *GlobalVolumeFade* (a variable the program can use to fade the music) and *No songline start* (the song always starts from song line 0). The text box lists the

*RMT FEATures* the song uses: if a game uses only this song, these definitions can be pasted into the player routine source (`rmt_feat.a65`), which then assembles to a smaller and faster routine (this is described in the file `optimization.txt` that comes with the program).

### 12.2.2 ASM simple notation

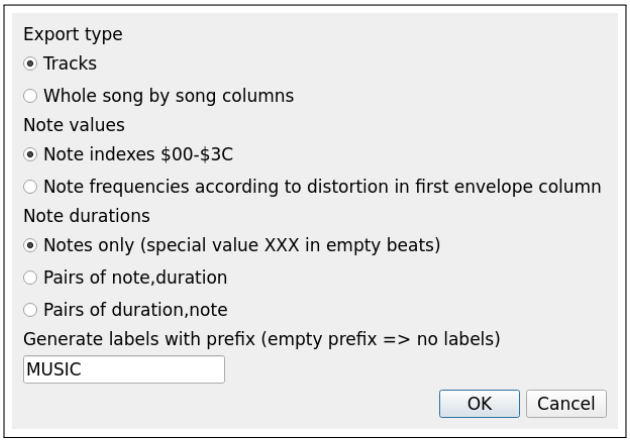



**Figure 12.5.** The export of the notes as assembler data.

Choose what to export (the tracks, or the whole song by song columns), how the note is written (the index `$00`–`$3C`, or the frequency according to the distortion of the instrument), how the duration is written (notes only, with the special value `XXX` in empty beats, or pairs of note and duration, in either order) and the prefix of the labels (empty for none).

### 12.2.3 SAP, SAP-R and LZSS

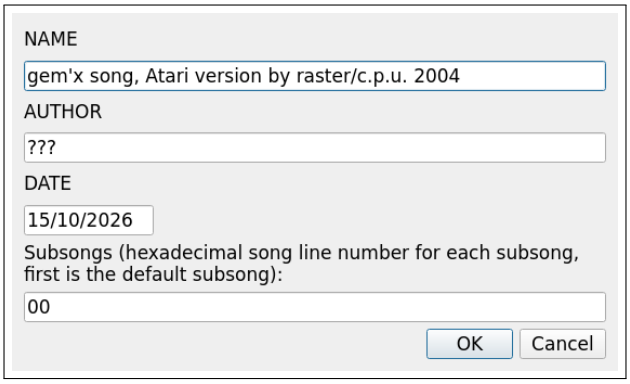



**Figure 12.6.** The export of a SAP file.

A SAP file has a header with the name, the author and the date, which the dialog fills with those of the song. The field *Subsongs* lists, in hexadecimal, the song line each subsong starts from: `00` for one tune, `00 10` for two. SAP-R is the stream of the POKEY registers; LZSS compresses it, and the SAP file contains it with the player routine of the VU-Player.

In the SAP file each subsong plays its intro once and then its loop for ever, at full volume. The SAP files of RITMO before 2.6 did not: at the second end of the loop the player started a fade

out that stopped after one step, so from the third pass on the song played one volume step lower and one frame early.

### 12.2.4 XEX

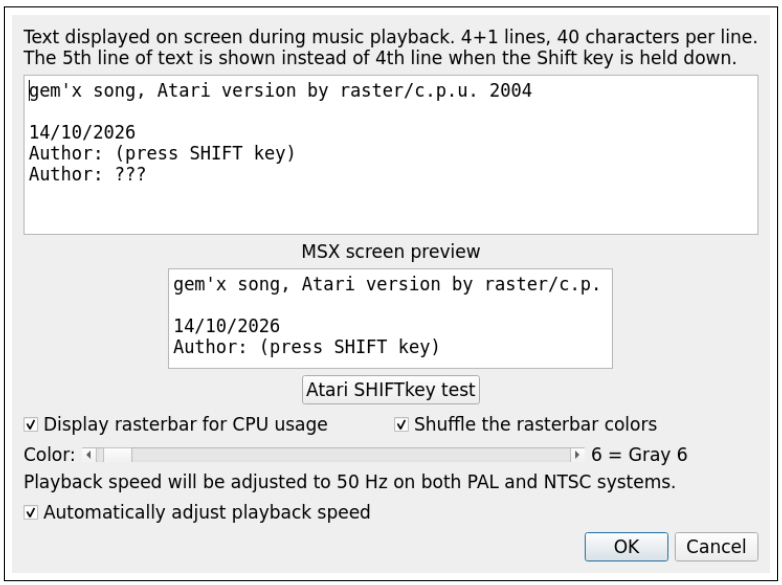



**Figure 12.7.** The export of an Atari executable.

The program an XEX file contains shows a text on the Atari screen while it plays: up to five lines of 40 characters (the fifth shown instead of the fourth when the SHIFT key of the Atari is held down). Below, the *rasterbar* shows the CPU usage of the player in colour, *shuffle* changes its colours, and the box *Automatically adjust playback speed*, with the line that says that the speed will be adjusted to 50 Hz on both PAL and NTSC systems, turns the adjustment of the speed on. The preview shows the text as the Atari will draw it.

### 12.2.5 XEX with the RMT player

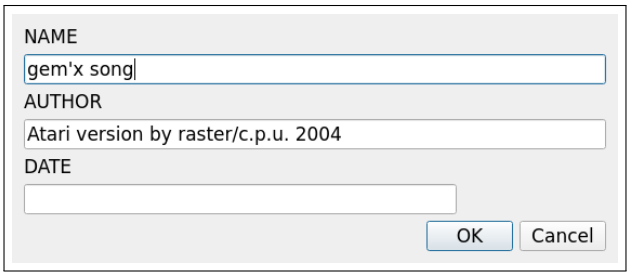



**Figure 12.8.** The export of an Atari executable with the RMT player.

This XEX holds the RMT player and the module of the song, not the stream of the POKEY registers: the file is smaller, a long song or a stereo song at a high instrument speed fits, and the Atari computes the music as the tracker does. The player is the one of the tracker driver: RMT 1.28 for *RMT 1.28 Unpatched*, Patch16 for *RMT 1.34 Patch16*, with the frequency tables of the tuning in use. With the other drivers there is no player and the export stops with a message.

The dialog asks for the three lines the program shows, NAME, AUTHOR and DATE (up to 200 characters each; a line longer than the 40 characters of the screen scrolls). It proposes them from the name of the song, as the tracker writes it: the name up to its first comma, the rest, and the first year it mentions.

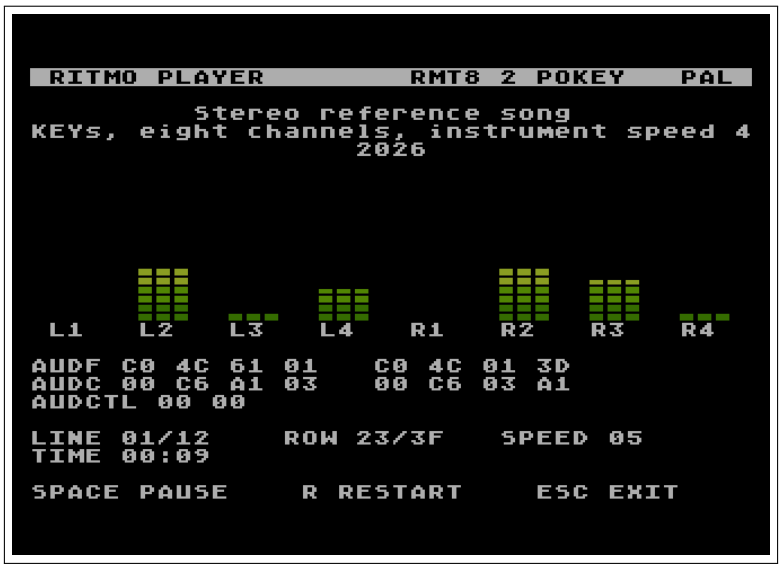



**Figure 12.9.** RITMO PLAYER on the Atari, with the stereo reference song.

On the Atari, RITMO PLAYER (the visualizer of RMTPLAY of VERA_ATARI_PBI) shows the three lines, one volume bar per channel, the POKEY registers, the song line, the row, the speed and the play time; `SPACE` pauses, `R` starts the song again and `ESC` goes back to DOS. An 8-track song plays on two POKEYs, with eight bars. The instrument speed (1 to 4) is the number of calls of the player in every frame; a song for the other region (PAL or NTSC) plays at its own speed. The module goes after the program, up to `$B820`: about 24 KB; a bigger one stops the export with a message.

### 12.2.6 Relocatable ASM

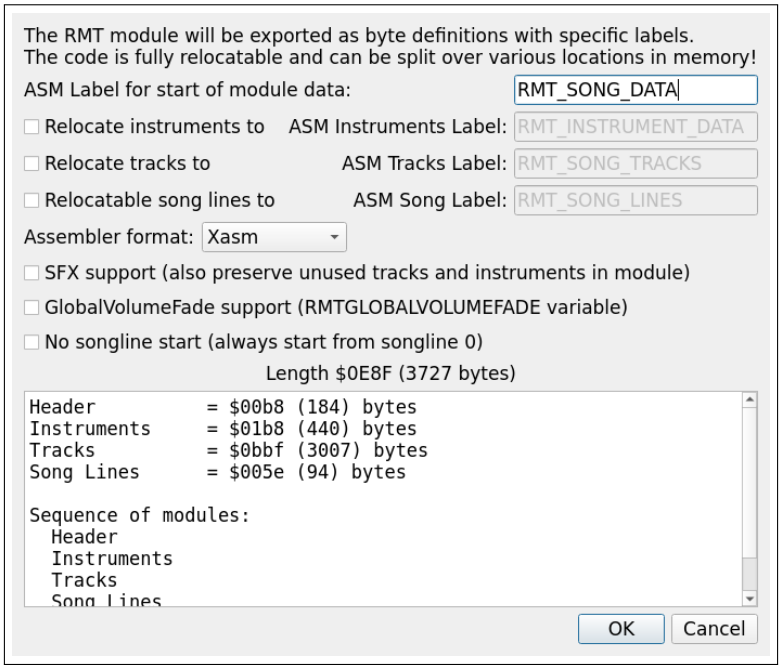



**Figure 12.10.** The export of a relocatable module.

The module is written as byte definitions with labels (the one of the start of the module data, and optionally separate labels for the instruments, the tracks and the song lines, which can be placed in different places of memory). The text box shows the size of each part and the order of the modules.

The *Assembler format* is *Atasm*, *Xasm* (also MADS) or *CA65 (cc65)*, the assembler of the cc65 suite. In the CA65 format the module is one `.proc` named by the start label, which is exported: its own labels do not collide with those of the program or of another song, and the linker places it. The parts placed at labels of your own begin with `.org ::`*label*: define those labels as constants (`ca65 -D RMT_SONG_LINES=$7000`). The feature section at the top (the `FEAT_` lines, to make the player smaller) is written as `FEAT_… = 1` for `rmt_feat.s`.

### 12.2.7 The RMT player in a C program (cc65)

A song can play in a program written in C with cc65. The folder `asm/cc65-example` of the repository is a complete example, with its `README.md`: `make` builds an XEX that plays a song and shows the frames played. It has three parts:

**The player**
: the Patch16 RMT player of `asm/ca65`, assembled by ca65 at `$3100` with its frequency tables at `$3A00` and its variables from `$2D40` (zero page `$CB-$DD`). It is not relocatable: it is assembled on its own and appended to the XEX of the program, so DOS loads it before the program runs.

**The module**
: the song, exported as *Relocatable ASM for RMTPlayer* in the CA65 format, without relocation. With the start label `_rmt_module` it is `extern const unsigned char rmt_module[];` in C.

**The program**
: the C code and a small wrapper (`rmt.s`, `rmt.h`): `rmt_init(rmt_module)` starts the song, `rmt_vbi_on()` puts the player on the deferred VBI (it calls it as many times per frame as the instrument speed), `rmt_vbi_off()` stops it. It is linked from `$4000` with `atari-rmt.cfg`, the `atari.cfg` of cc65 changed to leave the player its memory.

The player has to match the tracker driver of the song: Patch16 for the default driver of RITMO, the RMT 1.28 player (`asm/ca65/Legacy/rmt_xex.s`) for a song made with *RMT 1.28 Unpatched*. An 8-track song needs the player for two POKEYs (`make STEREO=1`). RITMO PLAYER, the program of the XEX export with the RMT player (`asm/rmtplay`), is a bigger example of the same kind.

### 12.2.8 WAV

The WAV export has no options: it plays the song with the built-in emulation and writes the sound.

## 12.3 Exporting from the command line

All the formats can be exported from a script, with no window, so that the files of a release are made from the `.rmt` source (chapter 16).

CHAPTER 13

# MIDI input

RITMO plays and records notes from a MIDI keyboard. The input device is chosen in **Tools** → **Options** (the MIDI group); the button **Toggle MIDI on/off** of the toolbar switches the input on and off while a device is set. The device is a port of the system: on Linux the ports of the ALSA sequencer (a USB keyboard or a virtual port), on Windows WinMM, on macOS CoreMIDI.

## 13.1 Options

| Option | Meaning |
|---|---|
| MIDI IN device | the input device, or *none* |
| Touch response | the key velocity becomes the volume of the note: `Atari Volume Offset + velocity / 8`, limited to 1–15. Without it the current volume (as for the typed notes) is used |
| Atari volume offset | added to the volume of the velocity, from 0 to 15 |
| Record note off | a released key (a note off, or a note on with velocity 0) deletes the note it played and writes volume 0. Without it releases are ignored |

## 13.2 The MIDI channels

| Channel | Function |
|---|---|
| 1 | Records notes at the cursor, like the note keys: the MIDI note 36 is the lowest note of the tracker (`C-1`), the note is written with the active instrument, the cursor moves down by the note spacing, and the note is played. While the song plays with follow mode, a note in the first half of a line is written on the next line (quantized). Outside the tracks area, in a jam mode or with `SHIFT` or `CONTROL` held down, the note is only played. Recording needs the window to have the focus |
| 2–9 | Live play on the channels `L1`–`L4` and `R1`–`R4` (6–9 repeat `L1`–`L4` on a mono song): never recorded, and independent of the focus. The velocity is the volume (velocity / 8); a program change sets the instrument of the channel; the controller 123 (all notes off) stops its note and the controller 121 (reset all controllers) restarts the player routine |
| 11–15 | a program change chooses the active instrument |
| 10 and 16 | an experimental assignment of controllers, kept from the original program, with the `POKEY explorer` (chapter 14) on channel 16 |

A system reset message (`FF`) restarts the player routine and forgets the notes held on the channels 2 to 16. The implementation chart of the MIDI messages is in appendix C.

## 13.3 Scripts

A script can send MIDI messages to the program with the `midi` command, which is how the recording can be tested without a keyboard (chapter 16).

CHAPTER 14

# The Pokey explorer

The explorer is a mode to study the sound of the POKEY. **Pokey → Activate Pokey Explorer Mode** (`CONTROL+SHIFT+F5`) stops the tracker's player routine and lets the keys write the registers of the chip directly, so that you can hear the effect of each divisor, distortion and `AUDCTL` bit. The mode is left with the edit/jam toggle (`CONTROL+SPACE`).

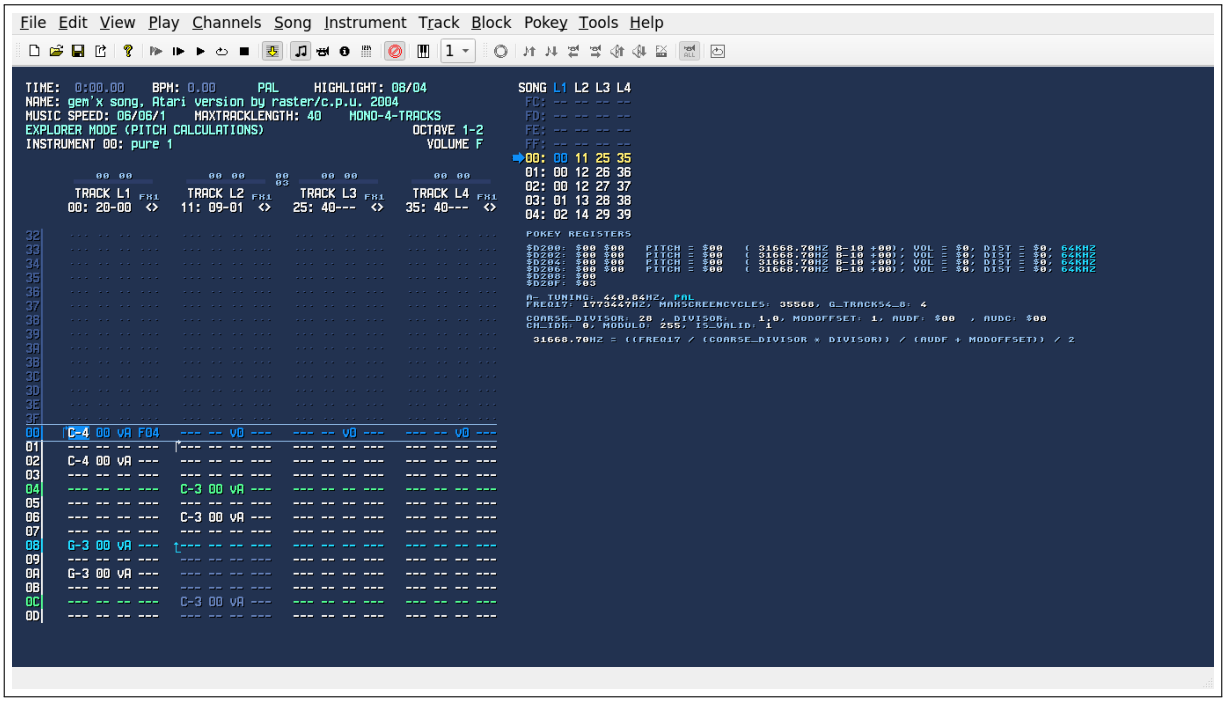



**Figure 14.1.** The Pokey explorer mode, with the registers of the chip on the screen.

With **Pokey → Pokey Chip Registers** on, three rows under the dumps of the registers detail one channel: its `AUDF` and `AUDC` bytes, the first divisor of `AUDF + MODOFFSET` from 3 up (`MODULO`), the coarse divisor of the pitch formula (28 at 64 kHz, 114 at 15 kHz, 1 at 1.79 MHz and for joined channels), the free divisor, the modulo offset (1, 4 at 1.79 MHz, 7 for joined channels) and the resulting pitch.

## 14.1 The keys

The keys are the physical positions of the QWERTY layout (on QWERTZ and AZERTY they are the keys at the same positions).

| Key | Function |
|---|---|
| ENTER / BACKSPACE | next / previous channel (0–3) |
| + / - | divisor +0.1 / −0.1; with SHIFT +1.0 / −1.0 (from 1.0 to 10000.0) |
| 1 3 5 7 | AUDF0 to AUDF3 +1; with SHIFT +$10 |
| Q E T U | AUDF0 to AUDF3 −1; with SHIFT −$10 |
| 2 4 6 8 | AUDC0 to AUDC3 +1; with SHIFT +$10 |
| W R Y I | AUDC0 to AUDC3 −1; with SHIFT −$10 |
| C G F K J D A P | toggle the bits 0 to 7 of AUDCTL: the 15 kHz clock, the two high pass filters, the two 16-bit joins, the two 1.79 MHz clocks and the 9-bit poly counter |
| M | toggle the two-tone bits of SKCTL |

The **Pokey** menu has the same operations as menu items, which work in the explorer mode only.

CHAPTER 15

# Tracker drivers

## 15.1 Tracker Drivers

A tracker driver is part of the tracker that generates the actual sound from the tracker data and user input. In the case of RMT, the tracker diver is a piece of MOS 6502 code that runs on an emulated Atari 8-bit computer.

## 15.2 Tracker Driver Versions

A tracker driver version is a variant of the tracker driver that interprets the data and user input differently. For example, different tracker driver versions have a distinct feature set or tuning. A correct replay of the music requires using the same tracker driver version that was used to create it.

The original tracker driver version is the one by Raster in RMT 1.28. Many people modified the driver code and data to yield new effects. All known older versions are preserved for compatibility.

Tracker driver versions are identified in different ways. For inclusion into RMT, the tracker driver versions are numbered starting from 1 (0 is no driver). The number is the one of `TRACKERDRIVERVERSION` in `rmt.ini`, of the constant in the source code (`TrackerDriverVersion.h`) and of the binary `resources/drivers/rmt_driver_v<code>.obx`. Additionally, they have a readable name for users (the "RMT Driver Version" list of the module properties, File > Properties) and a name for scripts (`set driver`, see `rmt_scripting.md`).

The level of a driver is the RMT version whose driver code it is: the original by Raster or a patch of it.

| Code | Name in the module properties | Level (driver code) | Script name | Constant name | Author |
|---|---|---|---|---|---|
| 0 | No RMT Driver | - | - | NONE | - |
| 1 | RMT 1.28 Unpatched by Raster | RMT 1.28, original | `unpatched` | UNPATCHED | Raster |
| 2 | RMT 1.28 Unpatched with Tuning | RMT 1.28, original (same binary as 1) | `unpatched-with-tuning` | UNPATCHED_ WITH_TUNING | VinsCool |
| 3 | RMT 1.25 Patch3 Instrumentarium by Analmux | RMT 1.25, Patch 3 | `patch3` | PATCH3 | Analmux |
| 4 | RMT 1.27 Patch6 by Analmux | RMT 1.27, Patch 6 | `patch6` | PATCH6 | Analmux |
| 5 | RMT 1.28 Patch8 by Analmux | RMT 1.28, Patch 8 | `patch8` | PATCH8 | Analmux |

| Code | Name in the module properties | Level (driver code) | Script name | Constant name | Author |
|---|---|---|---|---|---|
| 6 | RMT 1.34 Patch16 by VinsCool | RMT 1.28, Patch16-3.2 (RMT 1.31 and later; the default since RMT 1.34) | `patch16` | PATCH16 | VinsCool |
| 7 | RMT 1.28 Patch Prince of Persia by VinsCool | RMT 1.28, Patch 8 modified | `patch-prince-of-persia` | PATCH_ PRINCE_OF_ PERSIA | VinsCool |

The default is 6, Patch16.

Compatibility with the AUTOFILTER of RMT 1.28 (measured on a song that uses the AUTOFILTER: the frequency offset of channel 3 against channel 1 is the same as with the original RMT 1.28 player):

| Code | AUTOFILTER as RMT 1.28 |
|---|---|
| 1, 2, 3, 4, 5, 7 | yes |
| 6 | no (see RMT 1.34 Patch16 by VinsCool) |

### 15.2.1 RMT 1.28 Unpatched by Raster

The original RMT driver version that set the standard for many years with Atari 8-bit music. This version was mainly with RMT 1.28, as well as the unofficial RMT 1.30 by Rudla.

### 15.2.2 RMT 1.28 Unpatched with Tuning

This driver version by VinsCool was never completed: the binary `rmt_driver_v2.obx` is the same as `rmt_driver_v1.obx`, so it plays exactly as RMT 1.28 Unpatched. It is listed so that a configuration or a script that names it keeps working.

### 15.2.3 RMT 1.25 Patch3 Instrumentarium by Analmux

Adapted from Analmux's original post on AtariAge: > The music track ”Instrumentarium” is composed, tracked, and RMT-patched by me: ANALogue MUltipleXer. I used a patch to change some RMT possibilities: f.e, I added sawtooth wave (ringmod), triangle wave (linear subharmonic), clarinet (16-bit), distortion guitar (16-bit), and sync modes (2-tone filter & 16-bit). At the end of 2009, this tune was named ”Instrumentarium Remix 1”; it's the same tune. I was using exotic undocumented POKEY features. It takes almost no CPU time.

### 15.2.4 RMT 1.27 Patch6 by Analmux

Adapted from Analmux's original post on AtariAge: > Tables freqtabbasslo and freqtabbasshi are patched (all I did was multiply each 16-bit value by 15). The '16-bit distortion 6' instrument, which was really a generator C instrument, is now a generator A instrument. I removed the blocking of the 1st channel, to enable 16-bit filter-like effects.”

### 15.2.5 RMT 1.28 Patch8 by Analmux

Adapted from Analmux's original post on AtariAge:

| RMT Distortion | POKEY Distortion | Bits | Clock | Number of Bytes | Instrument |
|---|---|---|---|---|---|
| 0 | 0 | 8 | x | x | White noise |
| 2 | 2 | 8 | 1.79 MHz | 48 | Poly 5 / Generator 2 |
| 4 | A | 8 | 1.79 MHz | 36 | Sawtooth / Triangle |
| 6 | C | 8LSB | 1.79 MHz | 32 | Clarinet (Poly 4), LSB |
| 8 | A | 8HSB | 1.79 MHz | 64 | Harmonic square undertones of Clarinet / Distortion guitar, HSB |
| A | A | 8 | 15 kHz | 48 | Pure bass |
| A | A | 8 | 1.79 MHz | 12 | Pure flute |
| C | C | 8 | 1.79 MHz | 31 | Poly 4 (degenerate 1) |
| C | C | 8 | 1.79 MHz | 17 | Poly 4 (degenerate 3) |
| E | 8 | 8LSB | 1.79 MHz | 32 | Distortion guitar (Poly 9), LSB |

### 15.2.6 RMT 1.34 Patch16 by VinsCool

This driver version is an extensive hack of the original RMT 1.28 driver by Raster, inspired directly by Analmux's RMT Patch8 hack. The new features include: more tuning tables, a two-tone filter toggle, an AUDCTL timing fix, AUTOFILTER behavior tweaks, etc. Unfortunately, this patch also introduced many compatibility issues with older modules and a couple of bugs. Despite that, this is probably the most ”feature-rich” version of the RMT driver patches created so far. All that came at the cost of extra memory usage and heavier CPU load, so this is no longer an ”optimal” driver. This is currently the default RMT driver version included and used by RMT 1.34 and later. The driver code is version Patch16-3.2 (`asm/Patch-16/rmtplayr.a65`), used by RMT since version 1.31; the name in the module properties says RMT 1.34, the version that made it the default.

The AUTOFILTER is one of these compatibility issues: Patch16 starts the frequency offset of channel 3 at 0 instead of 1 and does not accumulate the command 6, so the two channels of the filter stay in phase and an older song sounds quiet and an octave higher. A song file does not say which driver it was made with, so when the driver is Patch16 and a song that uses the AUTOFILTER is opened, RMT asks whether to play it with the RMT 1.28 Unpatched driver. The answer is for that song only: a new song or the next song opened uses the default driver of the module properties again. A script (`rmt /SCRIPT:`) answers No; it chooses the driver with `set driver`.

#### Tuning

Selecting this driver enables the tuning settings (File > Properties... > Tuning...); all other drivers play their own built-in frequency tables, so the button is disabled for them. The settings belong to the song, but are kept in `tuning.ini` (the settings of the user, see BUILD.md), not in the module.

| Setting | Default | Description |
|---|---|---|
| A4 Frequency | 440.83751645933 Hz for PAL / 444.8957788679134 Hz for NTSC | The reference pitch of the note A-4, in Hertz, in every key. The defaults are RMT's traditional tuning, not concert pitch 440 Hz: the frequency tables the Patch16 player ships were generated with exactly these values (`RmtCoreTest --tuning` checks that equality). The hand-made tables of the RMT 1.28 era drivers are close to, but not byte-identical with, the generated ones. |
| Tonic | A- (3) | The key the tables are tuned to: the note that receives the temperament's 0 cents. Stored as a number, C- = 0 counting downward, so A- = 3. In equal temperament every tonic gives the same tables. |
| Temperament | Equal Temperament (Default) | Equal temperament, one of 21 historical presets (each verified against its source file in the Scala scale archive), or a custom temperament entered in musical cents (0-1200 for each semitone above the tonic). |

The two default A4 frequencies are in the exact ratio of the NTSC and PAL machine clocks (1789773 / 1773447) and therefore produce bit-identical frequency tables: every generated pitch is proportional to the A4 frequency, and every POKEY divider is computed from the quotient clock / pitch, so scaling the A4 frequency by the clock ratio cancels out exactly - before any rounding. This holds for every temperament, tonic and custom tuning, not only for equal temperament. It is why one player binary serves both systems, and why switching a module between PAL and NTSC multiplies the A4 frequency by exactly this ratio.

`tuning.ini` writes `A4_FREQUENCY`, `TONIC`, `TEMPERAMENT` and the eleven cents `MIN_2ND` … `MAJ_7TH`. A file of an older version is still read: `TUNING` and `BASENOTE` are the old names, a fraction (`MAJ_3RD = 5 / 4`) is converted to cents, and a preset that was retired (they were no 12-note temperament with a 2:1 octave) plays as equal temperament.

### 15.2.7 RMT 1.28 Patch Prince of Persia by VinsCool

This driver version is an older patch based on the original RMT 1.28 Patch 8 by Analmux. This version was created exclusively for the Prince of Persia soundtrack. It was thus never really used outside of a very few test tunes that used some of the sounds.

CHAPTER 16

# Scripting

RITMO can run a script file from the command line: open a module file, set what the export dialogs would ask, export in any format, save. This is meant for batch files, build scripts and CI, where the SAP, XEX, WAV and ASM files of a song are produced from its `.rmt` source reproducibly, and for repeated export sets while composing. The script format is the one of the Windows `Ritmo.exe` and of the Java port of the RASTER-Music-Tracker repository, so the same script runs in all of them.

## 16.1 Running a script

```
ritmo /SCRIPT:build.rmtscript
```

`-script:<file>` and `--script=<file>` are accepted too. The program starts without showing its window and exits with the script's exit code. The configuration (`rmt.ini` and `tuning.ini`, kept in the user's settings - `~/.config/ritmo-atari.org/ritmo.conf` on Linux) is read as for the window, so the tuning shapes the exports exactly as in the tracker; it is not written back.

- **Linux:** no display is needed - without `QT_QPA_PLATFORM` set, a script run uses Qt's `offscreen` platform. Nothing is played: the sound output is not opened.
- **Windows:** `rmt.exe` is a windowed program, so `cmd.exe` returns to the prompt at once unless it is started with `start /wait` (in PowerShell: `Start-Process rmt.exe "/SCRIPT:build.rmtscript" -Wait`).
- **macOS:** `Ritmo.app/Contents/MacOS/ritmo /SCRIPT:build.rmtscript`.

Messages that would be boxes in the window are printed to the console, all of them to stdout (on Windows a windowed program has no stderr that could reach the log file); a question is answered "No". On Windows, when there is no console (started from Explorer or a scheduler), they go to `<script>.log` next to the script.

Exit codes of the command line:

| Code | Meaning |
|---|---|
| 0 | every command succeeded |
| 1 | a command failed; the script stopped there and the message names the line |
| 2 | the script file could not be read or has a syntax error (the message names the line) |

## 16.2 Syntax

- One command per line. Blank lines are ignored, `#` starts a comment (at the start of a line or before a token).

- Tokens are separated by blanks. A value with blanks is quoted: `"My Song.sap"`. Inside quotes, `\"` is a quote and `\\` a backslash.
- Options are written `name=value` after the command's arguments, in any order: `export sap out.sap author="Raster" subsongs=0`. A quoted value works after the `=` as well.
- Command names, option names and option values that are choices (`yes`/`no`, format names, driver names) are case-insensitive; other values (texts, labels, file names) keep their case.
- Yes/no options accept `yes`, `no`, `true`, `false`, `on`, `off`, `1`, `0`.
- Relative file names are relative to the script file's folder, so a script travels with its module files. Output files (`save`, `export`) go to the folder of `set output` instead when the script gave one; the folder is created when it is missing.
- An output file that exists already is an error, unless the script says `set overwrite yes` first.

## 16.3 Commands

| Command | Meaning |
|---|---|
| `open <file>` | Opens a module file. The format follows the extension: `.rmt`, `.txt` or `.rmw` (as File > Open). |
| `save <file>` | Saves the module as `.rmt`, `.txt` or `.rmw` by extension (as File > Save as). The RMT format's integrity check applies. |
| `export <format> <file> [name=value ...]` | Exports the module (as File > Export as). Formats and options below. |
| `set overwrite yes\|no` | Whether later `save`/`export` commands may replace existing files. Default `no`. |
| `set output <folder>` | Where later `save`/`export` commands write (relative to the script's folder; created when missing). Default: the script's folder. |
| `set ntsc yes\|no` | The NTSC setting of the module properties (the video standard: 60 or 50 frames per second, and the POKEY clock). |
| `set driver <version>` | The tracker driver of the module properties (the song's and the default): `unpatched`, `unpatched-with-tuning`, `patch3`, `patch6`, `patch8`, `patch16` (the default), `patch-prince-of-persia`. |
| `set midi-touch-response yes\|no` | The Options dialog's MIDI "Touch response" (the velocity becomes the volume). Default: `rmt.ini`'s. |
| `set midi-volume-offset <0-15>` | The Options dialog's MIDI "Atari Volume Offset". Default: `rmt.ini`'s. |
| `set midi-note-off yes\|no` | The Options dialog's MIDI "Record note off". Default: `rmt.ini`'s. |

| Command | Meaning |
|---|---|
| `midi <status> <data1> [<data2>]` | Handles a MIDI message given as hex bytes exactly as one from the MIDI IN device would be (see the manual's ”MIDI Input”): `midi 90 3C 64` is note on C-3, velocity 100, on channel 1 and records the note at the cursor; `midi CA 05` selects instrument 5. The window counts as focused. |
| `dump actions <file>` | Writes the program's command table - every menu item and toolbar button with its key and its prompt from `Rmt.rc` - as a Markdown table. The table is read from the running program's menus and toolbars, so it lists the keys the menu items really have (the tracker's own keys, handled in the editing code, appear only where a menu label shows them). A row marked ERROR is a menu label that shows another key than the item has and fails the command. |
| `dump notekeys <file>` | Writes the note keys of the QWERTY, AZERTY and QWERTZ keyboard layouts (which key plays which note) as keyboard pictures and Markdown tables . |
| `echo <text ...>` | Prints the text. |
| `quit` | Ends the script (implicit at its end). |

## 16.4 Export formats and options

The format names are those of the Export dialog's file-type list. Every option has the value the dialog would show when it opens, so a bare `export sap out.sap` produces exactly what the dialog's OK produces with untouched fields. The extension of the format is added to the file name when it is missing.

| Format | File | Options |
|---|---|---|
| `stripped-rmt` | `.rmt` | `address` (the module's address, `$4000` or decimal; default: the loaded module's), `sfx` (SFX support), `gvf` (global volume fade), `nos` (no starting songline), `asmformat` = `xasm`, `atasm` or `ca65` |
| `asm` | `.asm` | `type` = `tracks` or `song` (tracks, or the whole song by song columns), `notes` = `index` or `freq`, `durations` = `notes`, `note-duration` or `duration-note`, `prefix` (the label prefix, empty = no labels; default `MUSIC`) |
| `sapr` | `.sapr` | `author`, `name`, `date` (default: the song's name, the song's author, today), `subsongs` (default: the subsongs found in the song) |

| Format | File | Options |
|---|---|---|
| `lzss` | `.lzss` | none. Writes the full tune and, when they are longer than 16 bytes, the `_INTRO.lzss` and `_LOOP.lzss` siblings. |
| `sap` | `.sap` | as `sapr` (a SAP of type B with the LZSS player of VU-Player V2; each songline listed in `subsongs` starts a subsong, `subsongs="00 10"` makes two) |
| `xex` | `.xex` | `text` (the screen text, up to 5 lines of 40 characters, `\n` between lines; default: the song's name, date and author lines the dialog proposes), `rasterbar` (yes/no), `shuffle` (yes/no), `region-auto` (yes/no), `color` (the rasterbar color, 0-255) |
| `xex-rmt` | `.xex` | `name`, `author`, `date` (the three info lines of RITMO PLAYER, up to 200 characters each; default: the song name up to its first `,`, the rest, its first year; a module without a name gets its file name). An XEX with the RMT player of the tracker driver (`unpatched` or `patch16`; other drivers stop the script) and the module; an RMT8 song plays on two POKEYs. |
| `rmtplayer-asm` | `.asm` | `startlabel` (default `RMT_SONG_DATA`), `relocate` (a comma list of `instruments`, `tracks`, `songlines`, or `none`), `instruments-label`, `tracks-label`, `songlines-label`, `asmformat` (`xasm`, `atasm` or `ca65`: the CA65 source is a `.proc` named by `startlabel`, exported), `sfx`, `gvf`, `nos` |
| `wav` | `.wav` | none (16-bit, 44.1 kHz, 2 channels, the song once up to its loop point) |

Options that a format does not have, and values that are not valid, stop the script with a message naming the line.

## 16.5 Example

```
# Build the release assets of Delta
set overwrite yes
open Delta.rmt
export sap Delta.sap author="Raster" name="Delta" subsongs="0"
export xex Delta.xex text="DELTA BY RASTER\nJAVA PORT" rasterbar=no
export stripped-rmt Delta_stripped.rmt address=$5000
export wav Delta.wav
echo finished
```

```
ritmo /SCRIPT:Delta/build.rmtscript
```

## 16.6 Environment variables

These variables let a build script run one and the same script file with different outputs or settings, without editing it:

| Variable | Programs | Meaning |
|---|---|---|
| `RMT_SCRIPT_OUTPUT=<folder>` | all | The output folder for every `save`/`export`; overrides the script's `set output`. |
| `RMT_SCRIPT_LOG=<file>` | all | Writes the messages to this file instead of the console or `<script>.log`. |
| `RMT_SCRIPT_SHOW_WINDOW=1` | all | Shows the window during the script run (normally it stays hidden), to watch what a script does. |
| `RMT_CONFIG_DIR=<folder>` | all | Reads the configuration from `ritmo.ini` and `tuning.ini` in this folder (text files, as RMT wrote them) instead of the user's settings; a missing file means the defaults. The same in the window, which also saves them there. |

A script run reads the configuration and never writes it, not even the defaults on a machine that has none yet: `set` changes the settings of the run only.

## 16.7 Tests

`test-resources/scripts` holds two reference scripts (`delta.rmtscript`: the mono reference song through every save and export, once with the defaults and once with every option set; `midi.rmtscript`: MIDI messages recorded into it). `scripts/test-scripting.sh [path/to/ritmo]` runs both twice into separate folders and checks the exit codes, that the files exist and that the two runs are byte-identical; it also checks the exit codes 1 and 2. The scripts are the ones the Windows program and the Java port compare their exports with, so the output folders can be compared with those programs' as well (WAV files excepted: 8-bit in `Ritmo.exe`, 16-bit in the Java port).

## 16.8 Not available as script commands

Importing (MOD, TMC), the instrument files (`.rti`), the track files and `New` - the tracker's interactive functions - and, in this program, "Tools > Run Script" in the window. The command set follows the export workflow; further commands can be added if a need arises.

APPENDIX A

# Reference of the commands, keys and fields

This appendix is the complete reference: every command of the menus and the toolbars with its key, the note keys of the three keyboard layouts, and the keys and the fields of each part of the screen. The first two tables are written by the program itself.

Every command of the menus and toolbars, with its key and what it does. This table is generated from the program itself (the `dump actions` script command, see the scripting documentation; `scripts/make-docs.sh` regenerates it), so it is always the current state of the program.

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Menu File / Tool Bar Main | New | `Ctrl+N` | Create a new song |
| Menu File / Tool Bar Main | Open... | `Ctrl+O` | Load an existing song |
| Menu File | Reopen | `Ctrl+R` | Reload song from the last saved version |
| Menu File / Tool Bar Main | Save | `Ctrl+S` | Save the song |
| Menu File | Save As... | `Ctrl+Shift+S` | Save the song with a new filename |
| Menu File | Import... | | Import song from file |
| Menu File / Tool Bar Main | Export | | Export song to file |
| Menu File | Print... | `Ctrl+P` | Print the active view |
| Menu File | Print Preview | | Preview the page |
| Menu File | Print Setup... | | Change the printer and printing options |
| Menu File | Properties... | `Alt+Enter` | Open the module properties: what the module holds, its video standard, tracker driver and tuning |
| Menu File | Exit | `Alt+F4` | |
| Menu Edit | Undo | `Ctrl+Z` | Undo the last action |

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Menu Edit | Redo | `Ctrl+Y` | Redo the last action |
| Menu Edit | Clear Undo & Redo History | | |
| Menu Edit / Tool Bar Main | Edit Tracks | `F2` | Move to track edit view |
| Menu Edit / Tool Bar Main | Edit Instruments | `F3` | Move to instrument edit view |
| Menu Edit / Tool Bar Main | Edit Info | `Shift+F4` | Move cursor to Info edit |
| Menu Edit / Tool Bar Main | Edit Song | `F4` | Move cursor to song edit |
| Menu Edit / Tool Bar Main | Switch Edit Mode | `Ctrl+Space` | Edit/Jam mode toggle |
| Menu View | Main Toolbar | | Show or hide the toolbar |
| Menu View | Block Toolbar | | |
| Menu View | Status Bar | | Show or hide the status bar |
| Menu View | Play Time Counter | | |
| Menu View | Volume Analyzer | | |
| Menu View | Instrument Active Help | | |
| Menu Play / Tool Bar Main | Play From Bookmark | `Shift+F7` | Play song from bookmark position |
| Menu Play / Tool Bar Main | Play From Start | `F5` | Play song from start position |
| Menu Play / Tool Bar Main | Play | `F7` | Play song from current position |
| Menu Play / Tool Bar Main | Play And Loop Song Line | `F6` | Play and loop current tracks pattern |
| Menu Play / Tool Bar Main | Stop | `Esc` | Stop playing the song. Mute all sounds. |
| Menu Play / Tool Bar Main | Toggle Follow Mode | `F12` | Follow the currently playing position (turn on/off) |
| Menu Channels | Channel 1 | `Ctrl+1` | |
| Menu Channels | Channel 2 | `Ctrl+2` | |
| Menu Channels | Channel 3 | `Ctrl+3` | |

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Menu Channels | Channel 4 | `Ctrl+4` | |
| Menu Channels | Channel 5 | `Ctrl+5` | |
| Menu Channels | Channel 6 | `Ctrl+6` | |
| Menu Channels | Channel 7 | `Ctrl+7` | |
| Menu Channels | Channel 8 | `Ctrl+8` | |
| Menu Channels | Mute/Unmute Active Channel | `F9` | |
| Menu Channels | Mute/Unmute All Channels | `Shift+F9` | Mute / Unmute all channels |
| Menu Channels | Solo Active Channel | `Ctrl+F9` | Mute all channels except active channel / Unmute all channels |
| Menu Song | Copy Line | | |
| Menu Song | Paste Line | | |
| Menu Song | Clear Line | | |
| Menu Song | Set Bookmark | `F8` | Set a bookmark in the current song line |
| Menu Song | Clear Bookmark | `Ctrl+F8` | Clear the bookmark in the current song line |
| Menu Song | Delete current line | `Ctrl+U` | |
| Menu Song | Insert new empty line | `Ctrl+I` | |
| Menu Song | Insert new line with unused empty tracks | `Ctrl+J` | |
| Menu Song | Insert copy or clone of song line(s)... | `Ctrl+K` | |
| Menu Song | Insert new empty unused track to current song position | `Ctrl+T` | |
| Menu Song | Make a track duplicate to current song position | `Ctrl+D` | |
| Menu Song | Switch song between 4 or 8 channels... | | |
| Menu Song | Song columns' order change/copy/clear... | | |
| Menu Song | Change maximal length of tracks... | | |

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Menu Song | All size optimizations... | | |
| Menu Instrument | Copy | | |
| Menu Instrument | Paste | | |
| Menu Instrument / Paste special | Volume envelopes only | | |
| Menu Instrument / Paste special | Envelope parameters only | | |
| Menu Instrument / Paste special | Volume envelopes and Envelope parameters only | | |
| Menu Instrument / Paste special | Insert Volume envelopes and Envelope parameters to cursor position | | |
| Menu Instrument / Paste special | Volume L envelope only | | |
| Menu Instrument / Paste special | Volume R envelope only | | |
| Menu Instrument / Paste special | Volume R to L envelope only | | |
| Menu Instrument / Paste special | Volume L to R envelope only | | |
| Menu Instrument / Paste special | Table only | | |
| Menu Instrument | Cut | | |
| Menu Instrument | Delete | | |
| Menu Instrument | Info about current instrument... | | |
| Menu Instrument | Change all the instrument occurences... | | |
| Menu Instrument | Renumber all instruments... | | |
| Menu Instrument | Load instrument from file... | | |

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Menu Instrument | Save instrument as... | | |
| Menu Instrument | Clear all unused instruments... | | |
| Menu Instrument | All instruments cleanup... | | |
| Menu Track | Copy | | |
| Menu Track | Paste | | |
| Menu Track | Cut | | |
| Menu Track | Delete | | |
| Menu Track | Increase Step Size | `Num +` | Increase the pattern step size |
| Menu Track | Decrease Step Size | `Num -` | Decrease the pattern step size |
| Menu Track | Info about current track... | | |
| Menu Track | Search and build wise loop | | |
| Menu Track | Expand loop | | |
| Menu Track | Search and rebuild wise loops in all tracks... | | |
| Menu Track | Expand loops in all tracks | | |
| Menu Track | Renumber all tracks... | | |
| Menu Track | Load track from file... | | |
| Menu Track | Save track as... | | |
| Menu Track | Clear all duplicated tracks, adjust song... | | |
| Menu Track | Clear all tracks unused in song... | | |
| Menu Track | All tracks cleanup... | | |
| Menu Block / Tool Bar Block | Restore From Backup | `Ctrl+B` | Restore block from backup |
| Menu Block | Copy | `Ctrl+C` | |
| Menu Block | Paste | `Ctrl+V` | |
| Menu Block / Paste Special | Merge With Current Content | `Ctrl+M` | |
| Menu Block / Paste Special | Paste Only Volume Values | | |
| Menu Block / Paste Special | Paste Only Speed Values | | |
| Menu Block | Cut | `Ctrl+X` | |
| Menu Block | Delete | `Del` | |

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Menu Block | Exchange Block and Clipboard | `Ctrl+E` | |
| Menu Block / Tool Bar Block | Effects/Tools... | `Ctrl+F` | Effects/tools |
| Menu Block | Select All | `Ctrl+A` | |
| Menu Pokey | Pokey Chip Registers | | |
| Menu Pokey | Activate Pokey Explorer Mode | `Ctrl+Shift+F5` | Activate POKEY Explorer Mode |
| Menu Pokey / AUDCTL | Bit 0 - Change Main Base Clock From 64 KHz To 15 KHz | `C` | |
| Menu Pokey / AUDCTL | Bit 1 - High Pass Filter Into Channel 2, Clocked By Channel 4 | `G` | |
| Menu Pokey / AUDCTL | Bit 2 - High Pass Filter Into Channel 1, Clocked By Channel 3 | `F` | |
| Menu Pokey / AUDCTL | Bit 3 - Join Channels 3 and 4 (16-bit Frequency) | `K` | |
| Menu Pokey / AUDCTL | Bit 4 - Join Channels 1 and 2 (16-Bit Frequency) | `J` | |
| Menu Pokey / AUDCTL | Bit 5 - Clock Channel 3 With 1.79 MHz | `D` | |
| Menu Pokey / AUDCTL | Bit 6 - Clock channel 1 with 1.79 MHz | `A` | |
| Menu Pokey / AUDCTL | Bit 7 - Change The 17-Bit Poly To 9-Bit Poly (Only For Distortion 0 and 8) | `P` | |
| Menu Pokey / SKCTL | Two Tone Mode | `M` | |
| Menu Pokey / Debug Channel | Next Channel | `Enter` | |
| Menu Pokey / Debug Channel | Previous Channel | `Backspace` | |
| Menu Pokey / Divisor | Increase By 0.1 | `+` | |
| Menu Pokey / Divisor | Increase By 1.0 | `Shift++` | |
| Menu Pokey / Divisor | Decrease By 0.1 | `-` | |

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Menu Pokey / Divisor | Decrease By 1.0 | `Shift+-` | |
| Menu Tools | Open ASMA | | Open the Atari Sound Music Archive Website |
| Menu Tools | Open ASAP File... | | Open an .SAP or other Atari Sound Module File |
| Menu Tools | Run Script... | | Run an RMT script file (open, export, save) on the current session |
| Menu Tools | Options... | | Display the configuration menu |
| Menu Tools | Tuning... | | Display the tuning menu. |
| Menu Help | Help | `F1` | Open the user manual of RITMO in a browser |
| Menu Help | Online Help | `Shift+F1` | Open the reference of RMT (menus, keys, instruments) in a browser |
| Menu Help | About RITMO | | Display information about the app and its version in a pop-up dialog |
| Tool Bar Main | About | | Display program information, version number and copyright |
| Tool Bar Main | MIDI on/off | | MIDI on/off |
| Tool Bar Block | Transpose up | | Note transposition up |
| Tool Bar Block | Transpose down | | Note transposition down |
| Tool Bar Block | Change instrument | | Instrument number change |
| Tool Bar Block | Change instrument | | Instrument number change |
| Tool Bar Block | Volume up | | Volume up |
| Tool Bar Block | Volume down | | Volume down |

| Access Path | Entry | Accelerator Key | Action |
|---|---|---|---|
| Tool Bar Block | Block mode | | Block modification mode |
| Tool Bar Block | Play block | `Shift+F6` | Play selected block |
| | Toggle NTSC/PAL | `Ctrl+F12` | Toggle NTSC/PAL |

## A.1 Note Keys

The "tonekeys" of the hotkey tables below: two rows of the keyboard form a piano, the lower row from C-1 and the upper row from C-2, with the row above each as the black keys. The layout follows the "Keyboard layout" option (QWERTY, AZERTY or the German QWERTZ; a first start picks it from the keyboard language). The pictures and tables are generated from the program (the `dump notekeys` script command): each keyboard row as its keys print them, with the note every key plays underneath, then the same as a table by note.

### A.1.1 QWERTY

```
1    2    3    4    5    6    7    8    9    0    -    =
    C#2  D#2       F#2  G#2  A#2       C#3  D#3       F#3
  Q    W    E    R    T    Y    U    I    O    P    [    ]
 C-2  D-2  E-2  F-2  G-2  A-2  B-2  C-3  D-3  E-3  F-3  G-3
    A    S    D    F    G    H    J    K    L    ;    '
        C#1  D#1       F#1  G#1  A#1       C#2  D#2
      Z    X    C    V    B    N    M    ,    .    /
     C-1  D-1  E-1  F-1  G-1  A-1  B-1  C-2  D-2  E-2
```

| Note | Keys |
|---|---|
| C-1 | `Z` |
| C#1 | `S` |
| D-1 | `X` |
| D#1 | `D` |
| E-1 | `C` |
| F-1 | `V` |
| F#1 | `G` |
| G-1 | `B` |
| G#1 | `H` |
| A-1 | `N` |
| A#1 | `J` |
| B-1 | `M` |
| C-2 | `Q`, `,` |
| C#2 | `2`, `L` |
| D-2 | `W`, `.` |
| D#2 | `3`, `;` |

| Note | Keys |
|---|---|
| E-2 | E, / |
| F-2 | R |
| F#2 | 5 |
| G-2 | T |
| G#2 | 6 |
| A-2 | Y |
| A#2 | 7 |
| B-2 | U |
| C-3 | I |
| C#3 | 9 |
| D-3 | O |
| D#3 | 0 |
| E-3 | P |
| F-3 | [ |
| F#3 | = |
| G-3 | ] |

### A.1.2 QWERTZ

```
1    2    3    4    5    6    7    8    9    0    ß    ´
    C#2  D#2       F#2  G#2  A#2       C#3  D#3       F#3
  Q    W    E    R    T    Z    U    I    O    P    ü    +
 C-2  D-2  E-2  F-2  G-2  A-2  B-2  C-3  D-3  E-3  F-3  G-3
    A    S    D    F    G    H    J    K    L    ö    ä    #
        C#1  D#1       F#1  G#1  A#1       C#2  D#2
      <    Y    X    C    V    B    N    M    ,    .    -
          C-1  D-1  E-1  F-1  G-1  A-1  B-1  C-2  D-2  E-2
```

| Note | Keys |
|---|---|
| C-1 | Y |
| C#1 | S |
| D-1 | X |
| D#1 | D |
| E-1 | C |
| F-1 | V |
| F#1 | G |
| G-1 | B |
| G#1 | H |
| A-1 | N |
| A#1 | J |
| B-1 | M |
| C-2 | Q, , |
| C#2 | 2, L |

| Note | Keys |
|---|---|
| D-2 | W, . |
| D#2 | 3, ö |
| E-2 | E, - |
| F-2 | R |
| F#2 | 5 |
| G-2 | T |
| G#2 | 6 |
| A-2 | Z |
| A#2 | 7 |
| B-2 | U |
| C-3 | I |
| C#3 | 9 |
| D-3 | O |
| D#3 | 0 |
| E-3 | P |
| F-3 | ü |
| F#3 | ´ |
| G-3 | + |

### A.1.3 AZERTY

```
&    é    "    '    (    -    è    _    ç    à    )    =
    C#2  D#2       F#2  G#2  A#2       C#3  D#3       F#3
  A    Z    E    R    T    Y    U    I    O    P    ^    $
 C-2  D-2  E-2  F-2  G-2  A-2  B-2  C-3  D-3  E-3  F-3  G-3
    Q    S    D    F    G    H    J    K    L    M    ù    *
        C#1  D#1       F#1  G#1  A#1       C#2  D#2
      <    W    X    C    V    B    N    ,    ;    :    !
          C-1  D-1  E-1  F-1  G-1  A-1  B-1  C-2  D-2  E-2
```

| Note | Keys |
|---|---|
| C-1 | W |
| C#1 | S |
| D-1 | X |
| D#1 | D |
| E-1 | C |
| F-1 | V |
| F#1 | G |
| G-1 | B |
| G#1 | H |
| A-1 | N |
| A#1 | J |
| B-1 | , |

| Note | Keys |
|---|---|
| C-2 | A, ; |
| C#2 | é, L |
| D-2 | Z, : |
| D#2 | ", M |
| E-2 | E, ! |
| F-2 | R |
| F#2 | ( |
| G-2 | T |
| G#2 | - |
| A-2 | Y |
| A#2 | è |
| B-2 | U |
| C-3 | I |
| C#3 | ç |
| D-3 | O |
| D#3 | à |
| E-3 | P |
| F-3 | ^ |
| F#3 | = |
| G-3 | $ |

## A.2 For All Edit Modes

### A.2.1 Hotkeys

| Key | Action |
|---|---|
| ESC | Stop playing song, mute all sounds. With the option ”ESC resets Atari sound” (on by default) the sound routines are reinitialised as well. |
| F2 | Switch to TRACK EDIT. |
| F3 | Switch to INSTRUMENT EDIT. |
| SHIFT+F4 | Go to INFO EDIT. |
| F4 | Go to SONG EDIT. |
| F5 | Play song from start. |
| F6 | Play song and loop current pattern. |
| SHIFT+F6 | Play and loop selection block. |
| F7 | Play song from current cursor position. |
| SHIFT+F7 | Play song from bookmark position (if set). |

| Key | Action |
|---|---|
| F8 | Set bookmark to current cursor position. |
| CONTROL+F8 | Clear bookmark. |
| F9 | Mute/Unmute current channel. |
| CONTROL+F9 | Solo current channel/Unmute all channels. |
| SHIFT+F9 | Mute/Unmute all channels. |
| F11 | RESPECT VOLUME mode on/off. |
| F12 | Turn on/off autofollow mode. |
| CONTROL+F12 | Toggle PAL/NTSC region. |
| PAGE UP, PAGE DOWN | Go to previous/next SONG line. In TRACK EDIT they move the cursor by the highlight step within the pattern track instead; use CONTROL+PAGE UP, CONTROL+PAGE DOWN for the SONG line there. |
| SHIFT+PAGE UP, SHIFT+PAGE DOWN | Go to next/previous subsong. |
| numblock / | Decrease octave for newly entered notes. |
| numblock * | Increase octave for newly entered notes. |
| numblock +, numblock - | Change pattern step size up/down. |
| CONTROL+numblock +, CONTROL+numblock -, SHIFT+numblock +, SHIFT+numblock - | Increase/decrease volume for newly entered notes. The mouse wheel over the VOLUME field of the song info does the same, the wheel over the OCTAVE field changes the octave. |
| SHIFT+tonekeys | Play note with current instrument and volume on currently active channel. |
| SHIFT+CONTROL+tonekeys | Play note with current instrument and volume on currently active channel in both stereo channels at once. |
| CONTROL+SPACE | Toggle between EDIT MODE and PROVE (JAM) MODE. |
| SHIFT | SHIFT is used during text input for either the SONG NAME or INSTRUMENT NAME, for uppercase and special characters. |
| CAPSLOCK | Toggle CAPSLOCK (indicated by "CAP" in the statusbar). CAPSLOCK also inverts the SHIFT key if it is held at the same time. |
| CONTROL+1-8 | Turn on/off the channel 1 to 8. |
| CONTROL+O | Open RMT module. |

| Key | Action |
|---|---|
| CONTROL+R | Reload RMT module. |
| CONTROL+S | Save RMT module. |
| CONTROL+SHIFT+S | Save RMT module under a new name. |
| ALT+ENTER | Open the module properties (video standard, tracker driver, tuning). |
| CONTROL+N | Create new RMT module. |
| CONTROL+P | Print. |
| CONTROL+Y | Redo last change. |
| CONTROL+Z | Undo last change. |
| SHIFT+LEFT, SHIFT+RIGHT | Change active instrument. |
| Media PLAY | Play song from start/Stop. |
| Media NEXT, Media PREVIOUS | Play song from current pattern and skip next/previous SONG line. |
| ALT+F4 | Exit RMT. |

## A.3 TRACK EDIT

### A.3.1 Fields

```
NNN TT vV FSS
```

| Field | Meaning |
|---|---|
| NNN | Notes, from C-1 to C-6. |
| TT | Instrument number, from $00 to $3F. If no note exists in highligthed row, the instrument column will behave like a note column instead. |
| vV | Volume, from $0 to $F Could also be used without a note or an instrument. ”v” is simply added to clearly distinguish the volume value between the other columns. |
| FSS | Speed, from $01 to $FF. Fxx is intended to be a speed command, this is currently placeholder strings. |

### A.3.2 Hotkeys

| Key | Action |
|---|---|
| UP, DOWN, LEFT, RIGHT, TAB, CTRL+TAB, SHIFT+TAB, PAGE UP, PAGE DOWN | Move cursor. |
| CONTROL+UP, CONTROL+DOWN | Go to next/previous SONG line. |
| CONTROL+PAGE UP, CONTROL+PAGE DOWN | Go to next/previous SONG line (the same as CONTROL+UP, CONTROL+DOWN). |

| Key | Action |
|---|---|
| CONTROL+LEFT, CONTROL+RIGHT | Change track pattern number at current possition. |
| HOME | Move cursor to the start of current pattern track, or the start of a ”wise loop”. |
| CONTROL+HOME | Set/clear the start position of a ”wise loop”. |
| END | Move cursor to the end of current pattern track, or the end of a ”wise loop”. |
| CONTROL+END | Set/clear the end line of a pattern track. |
| SHIFT+CONTROL+ENTER | Set/clear the end line of a pattern track (the same as CONTROL+END). |
| ENTER | Play note at the cursor position. Every new ENTER hotkeys will also loop inside a selection block, if it exists. |
| SHIFT+ENTER | Play note at the cursor position and fetch its instrument and volume values, then become the active instrument. |
| CONTROL+ENTER | Play all notes at the currently edited line. (The option ”Swap ENTER and CONTROL+ENTER” exchanges the two.) |
| CONTROL+D | Duplicate the current pattern track, and put it at the same place. (Note: If the pattern track is only used once in song, a messagebox asking for confirmation will appear.) |
| CONTROL+I, CONTROL+U | Insert/delete lines in the current pattern track. |
| INSERT, DELETE | Insert/delete lines in the current pattern track (the same as CONTROL+I, CONTROL+U). If a selection block exists, DELETE deletes its data instead. |
| CONTROL+G | Set ”go to line” command in the song at current position. |
| CONTROL+T | Put new empty unused track to current song position and active channel. |
| SPACE | Delete note, instrument, volume and speed values in the track at the cursor position. |

### A.3.3 Note Column

| Key | Action |
|---|---|
| tonekeys | Insert note and play it. (Note: If RESPECT VOLUME is active, previous volume value won't be replaced.) |
| numblock 1-6 | Change octave of the note at the cursor position and play it. |
| BACKSPACE | Delete note and instrument values (volume and speed values will not be cleared). |

### A.3.4 Instrument Number Column

| Key | Action |
|---|---|
| tonekeys | Insert note and play it. (Note: This will only work if the note column row is empty.) |
| 0-F | Set up/change instrument number. |
| BACKSPACE | Delete note and instrument values (volume and speed values will not be cleared). |

### A.3.5 Volume Column

| Key | Action |
|---|---|
| 0-F | Set up/change volume value. |
| BACKSPACE | Delete note, instrument and volume values (speed values will not be cleared). |

### A.3.6 Speed Column

| Key | Action |
|---|---|
| 0-F | Set up/change speed value. |
| BACKSPACE | Delete speed values. |

### A.3.7 Block Operations

| Key | Action |
|---|---|
| SHIFT+UP, SHIFT+DOWN, SHIFT+HOME, SHIFT+END | Select block data. |
| ESCAPE | Deselect block data. |
| DELETE | Delete selection block data. |
| HOME, END | Go to the start/end of the selection block. |
| CONTROL+A | Select all valid data from the currently edited pattern track, and form a selection block. |

| Key | Action |
|---|---|
| CONTROL+B | Restore selection block data from backup. The backup is created as soon as the selection block is being formed. |
| CONTROL+C | Copy selection block to the clipboard. If no selection exists, the data at cursor position will be taken. |
| CONTROL+E | Exchange of selection block data and clipboard data. |
| CONTROL+M | Paste and merge data from the clipboard to the cursor position. |
| CONTROL+V | Paste data from the clipboard to the cursor position. |
| CONTROL+X | Cut selection block to the clipboard. If no selection exists, the data at cursor position will be taken. |

### A.3.8 Block Data Modifications

| Key | Action |
|---|---|
| SHIFT+CONTROL+A | Block modifications mode. All changes are provided either on each line of the block, or only on lines with instrument number equal to active instrument. |
| CONTROL+F2, CONTROL+F1 | Transpose note up / down by semitones (5 octaves max). If no selection exists, the data at cursor position will be taken. |
| CONTROL+F4, CONTROL+F3 | Transpose note up / down by octaves (5 octaves max). If no selection exists, the data at cursor position will be taken. |
| SHIFT+CONTROL+LEFT, SHIFT+CONTROL+RIGHT | Change instrument values, from $00 to $3F max. If no selection exists, the data at cursor position will be taken. |
| SHIFT+CONTROL+UP, SHIFT+CONTROL+DOWN | Volume up/down, from $0 to $F max. If no selection exists, the data at cursor position will be taken. |
| CONTROL+F | Display the Block effects/tools window. |

### A.3.9 Mouse control

### A.3.10 Tracks top area (Shown as TRACK L1 to TRACK R4 above pattern tracks)

| Key | Action |
|---|---|
| `LeftMouseButton` | Turn channel on/off. |
| `RightMouseButton` | Solo channel/turn back on all channels. |

## A.4 INSTRUMENT EDIT

### A.4.1 Fields

| Field | Meaning |
|---|---|
| NAME | Name of the instrument, 32 chars max. |
| ENVELOPE LENGTH | Length of the envelope, from $01 to $20 (i.e. 32 max.). |
| ENVELOPE GOTO | Jump to given envelope position when the envelope end is reached. |
| FADEOUT | Volume slide when the end of envelope is reached for the first time. $00 = no volume slide, $FF = maximal volume slide. |
| VOL MIN | Minimal volume value reached by FADEOUT parameter. |
| TABLE LENGTH | Length of the table, from $01 to $20 steps (i.e. 32 max.). |
| TABLE GOTO | Jump to given table position when the table end is reached. |
| TABLE SPEED | Speed for the each table step, from $01 to $40 vbi. |
| TABLE TYPE | Table type. 0 = notes, 1 = frequencies |
| TABLE MODE | Table mode. 0 (SET) = add note (or frequency) to base note, 1 (ADD) = add note (or frequency) to the last calculated note (or frequency). Note: If the resulting note is outside of the C-1 to C-6 range (hex values $00 to $3D), the output volume will be zero. Frequency additions are not limited however. |
| EFFECT DELAY | Delay before the start of VIBRATO and FREQSHIFT effects by $01 to $FF vbi, $00 = no effects. |
| EFFECT VIBRATO | Vibrato effect, 3 preset levels from $01 to $03, $00 = no vibrato. |
| EFFECT FREQSHIFT | Frequency shifting effect, from $00 to $FF for each vbi. |
| AUDCTL 15KHZ | Turn on/off AUDCTL bit 0, ”Change main base clock from 64 KHz to 15 KHz”. |
| AUDCTL HPF 2+4 | Turn on/off AUDCTL bit 1, ”High pass filter into channel 2, clocked by channel 4”. |
| AUDCTL HPF 1+3 | Turn on/off AUDCTL bit 2, ”High pass filter into channel 1, clocked by channel 3”. |
| AUDCTL JOIN 3+4 | Turn on/off AUDCTL bit 3, ”Join channels 3 and 4 (16-bit frequency)”. |

| Field | Meaning |
|---|---|
| AUDCTL JOIN 1+2 | Turn on/off AUDCTL bit 4, "Join channels 1 and 2 (16-bit frequency)". |
| AUDCTL 1.79 CH3 | Turn on/off AUDCTL bit 5, "Clock channel 3 with 1.79 MHz". |
| AUDCTL 1.79 CH1 | Turn on/off AUDCTL bit 6, "Clock channel 1 with 1.79 MHz". |
| AUDCTL POLY9 | Turn on/off AUDCTL bit 7, "Change the 17-bit poly to 9-bit poly (Distortion 0 and 8 only)". |

### A.4.2 Table of Notes/Frequencies

| Field | Meaning |
|---|---|
| TABLE $00-$1F | From 1 to 32 steps per table. Values can be either notes (in semitones), or frequencies, based on the TABLE TYPE value. Values can range from positive ($00 to $7F) or negative (FF to $80) numbers. |

### A.4.3 Envelope

| Field | Meaning |
|---|---|
| VOLUME R | Volume values from $0 to $F, used only for stereo songs in tracks R1 to R4. |
| VOLUME L | Volume values from $0 to $F used in tracks L1 to L4. |
| DISTORTION | POKEY AUDC Distortion values, from $0 to $E. Only even values can be used, otherwise Volume Only output would occur. |
| COMMAND | Envelope command, from $0 to $7. |
| X, Y | Both parameters make use of values from $0 to $F for effect commands. They can be used as a two separate parameters, or as one 8-bit hexadecimal value $XY. |
| AUTOFILTER | Automatic High Pass Filter envelope effect. $0 = not active, $1 = active. It will only work with channels 1 or 2. This effect automatically enables the AUDCTL High Pass Filter bits for each channel, where HPF 1+3 or HPF 2+4 may be output. It has higher priority over non filtered voices, and will hijack them while the required conditions are met. If the resulting volume is zero, filtering won't be enabled, and won't take priority over other channels. Channels 3 and 4 won't be muted, so the resulting sound could be modulated into a richer tone if desired. |
| PORTAMENTO | Portamento. $0 = not active, $1 = active. If active, the "Portamento volatile frequency" is used instead of the current frequency. Portamento is mostly useful for producing pitch bending effects. |

### A.4.4 Sound Type (DISTORTION Parameter)

| Value | Meaning |
|---|---|
| 0 | Distortion 0, white noise. (AUDC $0v, Poly5+17/9) |
| 2 | Distortion 2, square-ish tones. (AUDC $2v, Poly5) |
| 4 | Distortion 4, no note table yet, Pure Table by default. (AUDC $4v, Poly4+5) |
| 6 | Distortion C, fallback setting, buzzy bass tones. (AUDC $Cv, Poly4) |
| 8 | Distortion 8, white noise. (AUDC $8v, Poly17/9) |
| A | Distortion A, pure tones. (AUDC $Av) |
| C | Distortion C, buzzy bass tones. (AUDC $Cv, Poly4) |
| E | Distortion C, gritty bass tones. (AUDC $Cv, Poly4) |

### A.4.5 Envelope command (COMMAND parameter)

| Value | Meaning |
|---|---|
| 0 | Play the base note shifted by $XY semitones. If the resulting note is outside the C-1 to C-6 range (hex values $00 to $3D), the volume output will be zero. |
| 1 | Play the frequency $XY directly. |
| 2 | Play the base note shifted by frequency $XY. |
| 3 | Add $XY semitones to base note. Play base note (new value). If the resulting note is outside the C-1 to C-6 range (hex values $00 to $3D), the volume output will be zero. |
| 4 | Add frequency $XY to FSHIFT register. Play base note. |
| 5 | Set up portamento speed $X, step $Y. Each $X vbi will be "volatile portamento frequency" shifted up or down by the $Y value. If $XY=$00, the current frequency will be used directly as volatile portamento frequency. |
| 6 | Add $XY value to FILTER_SHFRQ. Whenever a new note in track is playing, FILTER_SHFRQ is initialized to $01, so by default the Automatic Filter channels frequencies are offset by 1 unit in order to produce SID-like sound. |
| 7 | Set the instrument AUDCTL directly using $XY values. Play BASE_NOTE. Exceptions: $FF = VOLUME ONLY mode. $FE$/FD = enable/disable Two-Tone Filter (only for channel 1, modulated by channel 2). |

### A.4.6 Hotkeys

| Key | Action |
|---|---|
| `LEFT`, `RIGHT`, `UP`, `DOWN` | Move cursor. |
| `TAB` | Move cursor to the instrument parameters, the envelope parameters or the table parameters in succession |
| `SHIFT+TAB` | Move cursor to the instrument name line. |
| `0-F`, `CONTROL+LEFT`, `CONTROL+UP`, `CONTROL+RIGHT`, `CONTROL+DOWN`, `BACKSPACE`, `SPACE`, `DELETE` | Change parameter values (in valid ranges only). |

| Key | Action |
|---|---|
| HOME | In the parameters: move cursor to the first parameter (ENVELOPE LENGTH). |
| SHIFT+CONTROL+UP, SHIFT+CONTROL+DOWN | Change values for all steps of instrument envelope/table at cursor position. |
| SHIFT+CONTROL+numblock +, SHIFT+CONTROL+numblock - | Change the L+R volume envelopes up/down. If the cursor is on the ”VOLUME L” or ”VOLUME R” line, only the volume envelope for either line will be changed. |

### A.4.7 Table of notes/frequencies

| Key | Action |
|---|---|
| HOME | Move cursor to the start of the table/to the start of ”table loop”. |
| CONTROL+HOME | Set up the start of ”table loop”. |
| END | Move cursor to the end of the table. |
| CONTROL+END | Set up the end of the table. |
| INSERT | Insert empty step into the table at cursor position and shift to the right. |
| SHIFT+INSERT | Duplicate step at cursor position and shift to the right. |
| DELETE | Delete step at cursor position and shift to the left. |
| SPACE | Clear the table step at cursor position and move cursor to the right. |

### A.4.8 Envelope Parameters

| Key | Action |
|---|---|
| HOME | Move cursor to the start of the envelope/to the start of ”envelope loop”. |
| CONTROL+HOME | Set up the start of ”envelope loop”. |
| END | Move cursor to the end of the envelope. |
| CONTROL+END | Set up the end of the envelope/set up a maximal length of the envelope. |
| INSERT | Insert empty step into the envelope at cursor position and shift to the right. |
| SHIFT+INSERT | Duplicate step at cursor position and shift to the right. |

| Key | Action |
|---|---|
| DELETE | Delete step at cursor position and shift to the left. |
| SPACE | Clear the envelope step at cursor position and move cursor to the right. |

### A.4.9 Mouse control

### A.4.10 Volume Envelope Area

| Key | Action |
|---|---|
| LeftMouseButton | Draw the volume envelope. |
| RightMouseButton | Set the volume to zero in the envelope. |

## A.5 INFO EDIT

| Field | Meaning |
|---|---|
| NAME | Name of the song and author, 64 chars max. |

```
MUSIC SPEED: AA/MM/S
```

| Field | Meaning |
|---|---|
| AA | Current song speed, from $01 to $FF. |
| MM | Main speed, from $01 to $FF. By default the main speed is used when a song plays from the beginning. |
| S | Engine speed, from $1 to $8. (X player calls per frame.) Technically, speed above 4 is unsupported in the exported formats. |

### A.5.1 Hotkeys

| Key | Action |
|---|---|
| LEFT, RIGHT, TAB | Move cursor. |
| SHIFT+TAB | Move cursor song name line. |
| 0-F, CONTROL+LEFT, CONTROL+UP, CONTROL+RIGHT, CONTROL+DOWN | Change parameter values (in valid ranges only). |
| ENTER | Return from INFO EDIT to the currently active screen (Tracks or Instruments edit). |

### A.5.2 Mouse control

### A.5.3 MAXTRACKLENGTH, MONO/STEREO, PAL/NTSC

| Key | Action |
|---|---|
| `LeftMouseButton` | Toggle or prompt dialog boxes for each elements, then apply changes to the song. |

## A.6 SONG EDIT

| Key | Action |
|---|---|
| `LEFT`, `RIGHT`, `UP`, `DOWN`, `PAGE UP`, `PAGE DOWN`, `TAB`, `SHIFT+TAB` | Move cursor. |
| `HOME` | Move cursor to the start of the song. |
| `END` | Move cursor to last song line with valid data. |
| `SHIFT+PAGE UP` | Move cursor to the start of current subsong or start of previous subsong if the action is repeated. |
| `SHIFT+PAGE DOWN` | Move cursor to start of next subsong. |
| `0-F`, `CONTROL+LEFT`, `CONTROL+RIGHT`, `BACKSPACE` | Change the pattern track number or ”go to line” value in the song. |
| `INSERT`, `CONTROL+I`, `DELETE`, `CONTROL+U` | Insert/delete lines in the song (with auto-change of all relevant ”go to line” values). |
| `CONTROL+D` | Duplicate the current pattern track, and put it at the same place. (Note: If the pattern track is only used once in song, a messagebox asking for confirmation will appear.) |
| `CONTROL+G` | Set ”go to line” command in the song at current position. |
| `CONTROL+T` | Put new empty unused track to current song position and active channel. |
| `CONTROL+K` | Insert copy or clone of song line(s). |
| `CONTROL+J` | Prepare song line with unused empty tracks. |
| `ENTER` | Return from SONG EDIT to the currently active screen (Tracks or Instruments edit). |

## A.7 PROVE (JAM) MODE

Navigation is identical in TRACKS and SONG areas, but editing is disabled, and replaced with tonekeys always playing notes. INSTRUMENTS and INFO areas will still behave the same as they are in EDIT MODE, simply hold SHIFT to test notes, except when the text is being edited.

APPENDIX B

# The RMT module file format

The format of the `.rmt` files, for the programmers who read them. It is the format version 1, used by all the RMT trackers up to now; the description is the official one of the repository.

This is the module file format used by RMT trackers with version 1.xy.

### B.0.1 Segment Binary Load Header (Mandatory)

RMT module files are regular multi-segment binary load files. They start with a 6-byte binary load header. It includes the start address and end address in a virtual 64k address space. The absolute addresses in the segments and structures can be relocated based on their distance from the start address. > Offset Type Description ------ ---- ----------- $00 WORD Magic byte sequence $FFFF $02 WORD Start address, default is $4000 $04 WORD End address

The first segment holds the module, starting with the module header struct below. RMT saves its modules at $4000.

### B.0.2 Segment Names (Optional)

A module RMT saves has a second segment, which loads directly behind the first one. It holds the names, each terminated by a $00 byte:

| Type | Description |
|---|---|
| BYTE[] | Song name, at most 64 characters, terminated by $00 |
| BYTE[] | Name of the first instrument present, at most 32 characters, terminated by $00 |
| BYTE[] | Name of the next instrument present, ... |

The instruments present are the ones with an address other than $0000 in the instruments table, in the order of their numbers. Trailing spaces are not stored.

A module without this segment is what RMT calls ”stripped”: ”Export as Stripped RMT File” writes only the first segment, usually at another address and often with only the instruments and tracks the song uses, because a player needs nothing else. RMT loads a stripped module and keeps the default names.

### B.0.3 Module Header Struct

| Offset | Type | Description |
|---|---|---|
| $00 BYTE | [4] Head | er string ”RMT4” for mono or ”RMT8” for stereo modules |
| $04 BYTE | Trac | k length ($00 means 256) |
| $05 BYTE | Song | speed (number of frame before the next song line is played) |
| $06 BYTE | Engi | ne speed / Instrument speed (player calls per frame) |
| $07 BYTE | Form | at version number ($01 for player routine 1.x compatible format $02 for 2.x) |
| $08 WORD | Addr | ess of the instruments table |
| $0a WORD | Addr | ess of the low-byte table of the track addresses |
| $0c WORD | Addr | ess of the high-byte table of the track addresses |
| $0e WORD | Addr | ess of the track sequence table |

The format version $00 is the layout of RMT's pre-release versions, whose instruments are stored differently. RMT no longer reads such modules (since 2026-10-29). RMT 1.x writes $01.

### B.0.4 Order of the Structures

RMT writes the structures in this order, without gaps, so the address of each one follows from the sizes of the ones before it:

1. The module header struct.
2. The instruments table, with one entry per instrument number up to the highest instrument present.
3. The low-byte and then the high-byte table of the track addresses, with one entry per track number up to the highest track present.
4. The instrument structs, in the order of their numbers.
5. The track structs, in the order of their numbers.
6. The track sequence table, up to the end of the segment.

An absent instrument or track has the address $0000. A track's length is not stored: it ends where the next track present starts, and the last track ends where the track sequence table starts.

### B.0.5 Instrument Struct

| Offset | Type | Description |
|---|---|---|
| $00 BYTE | tlen | (pointer to the end of the table of notes) |
| $01 BYTE | tgo | (pointer to the loop of the table of notes) |
| $02 BYTE | elen | (pointer to the end of the envelope) |
| $03 BYTE | ego | (pointer to the loop of the envelope) |
| $04 BYTE | tspd | (bit 0-5), tmode (bit 6), ttype (bit 7) |
| $05 BYTE | AUDC | TL |
| $06 BYTE | Vsli | de |
| $07 BYTE | Vmin | (bit 4-7) |

| Offset | Type | Description |
|---|---|---|
| $08 BYTE | Dela | y ($00 for no vibrato & no fshift) |
| 09 BYTE | Vibr | ato |
| 0a BYTE | fshi | ft |
| 0b BYTE | Unus | ed |
| 0c | Tabl | e of notes |
| ?? | Enve | lope |

### B.0.6 Table of Notes Struct

| Type | Description |
|---|---|
| BYTE | Note or frequency (according to the type) |

### B.0.7 Envelope Struct

| Type | Description |
|---|---|
| BYTE | Volume (bits 0-3 for the left channel, bits 4-7 for the right channel (in RMT4 it's the same as bits 0-3)) |
| BYTE | Portamento (bit 0), distortion (bit 1-3), command (bit 4-6), filter (bit 7) |
| BYTE | XY |

### B.0.8 Track Struct

| Type | Description |
|---|---|
| BYTE | |
| bit 0-5 | note |
| bit 6-7 | volume (HI) or pause (1-3 beats) or special |

```
if note is $00-$3c:
BYTE
    bit 0-1 volume (LO)
    bit 2-7 instrument number
```

```
if note is $3d:
BYTE
    bit 0-1 volume (LO) volume only

if note is $3e:
    bit 6-7 pause
    if pause is $01-$03:    pause 1-3 beats
    if pause is $00:    next byte pause 1-255 beats

if note is $3f:
    if bit 6-7 is zero: next byte speed $01-$ff
    if bit 6 is zero, 7 is set up:  next byte is track jump pointer (go to $00-$ff from the
    beginning of track data)
    if bit 6-7 is set up:   END of track
```

A track ends with its end marker ($ff), with a jump (which is always its last event), or with a counted pause of 0 beats ($3e followed by $00). A track whose events fill all lines has no end marker. A track has at most 256 bytes; the jump's target is an offset within those bytes. The byte $7f (note $3f with only bit 6 set) is not an event.

### B.0.9 Instruments Table

| Type | Description |
|---|---|
| WORD | ptr_instr0 |
| WORD | ptr_instr1 |
| WORD | ptr_instr2 |
| ... | |

### B.0.10 Track Addresses Table

| Type | Description |
|---|---|
| BYTE | lowbyte_of_ptr_track0 |
| BYTE | lowbyte_of_ptr_track1 |
| BYTE | lowbyte_of_ptr_track2 |
| ... | |

```
BYTE    highbyte_of_ptr_track0
BYTE    highbyte_of_ptr_track1
BYTE    highbyte_of_ptr_track2
...
```

### B.0.11 Track Sequence Table

| Type | Description |
|---|---|
| BYTE | tracknumL1,tracknumL2,tracknumL3,tracknumL4, [tracknumR1,..,tracknumR4] |
| BYTE | tracknumL1,tracknumL2,tracknumL3,tracknumL4, [tracknumR1,..,tracknumR4] |
| BYTE | tracknumL1,tracknumL2,tracknumL3,tracknumL4, [tracknumR1,..,tracknumR4] |
| ... | |

```
if tracknum is FF, then an empty track is used

if tracknumL1 is FE, then gotoline(BYTE)=tracknumL2, goto_pointer(WORD)=(tracknumL3,4)
Note: gotoline(BYTE) is not used in the player (but the tracker uses it)
```

A ”go to” line therefore takes four bytes: $fe, the target line, and the target line's address, which is the address of the track sequence table plus the target line times the number of channels (4 or 8). In an RMT8 module the remaining four bytes of the line are $ff - except when the ”go to” line is the last line, where RMT stores only its four bytes and the segment ends there. When RMT loads a module, a ”go to” line whose target lies beyond the module's lines jumps to line 0.

APPENDIX C

# MIDI implementation chart

The MIDI implementation chart of the multitimbral instrument that the tracker is, from the original documentation of RMT (the program also accepts the controllers described in chapter 13).

```
************************************************************************
RASTER MUSIC TRACKER - RMT
by Radek Sterba, Raster/C.P.U., 2002-2008
http://raster.atari.org
************************************************************************

RMT [Raster Music Tracker] MULTITIMBRAL INSTRUMENT - MIDI IMPLEMENTATION CHART

---------------------------------------------------------------------------
Function              | Recognized           | Remarks
----------------------|----------------------|--------------------------------
Basic      Default    | 2-9                  | 2-5 = L1-L4 , 6-9 = R1-R4
Channel    Changed    | x                    | (Channel 1 is reserved for edit)
----------------------|----------------------|--------------------------------
Mode       Default    | Mode 4               |
           Messages   | x                    |
           Altered    | x                    |
----------------------|----------------------|--------------------------------
Note                  | 36-96                | RMT notes from C-1 to C-6
Number :   True voice | 36-96                |
----------------------|----------------------|--------------------------------
Velocity   Note ON    | o 9nH,v=1-127        | RMT volume = v / 8
           Note OFF   | x 9nH,v=0 8nH,v=xx   |
----------------------|----------------------|--------------------------------
After      Key's      | x                    |
Touch      Ch's       | x                    |
----------------------|----------------------|--------------------------------
Pitch Bend            | x                    |
----------------------|----------------------|--------------------------------
Control               | x                    |
Change                | x                    |
----------------------|----------------------|--------------------------------
Prog                  | o 0-63               | RMT instruments 00-3F
Change : True #       | o 0-63               |
----------------------|----------------------|--------------------------------
System exclusive      | x                    |
----------------------|----------------------|--------------------------------
Common :   Song Pos.  | x                    |
           Song Sel.  | x                    |
           Tune       | x                    |
----------------------|----------------------|--------------------------------
System   : Clock      | x                    |
Real Time: Commands   | x                    |
----------------------|----------------------|--------------------------------
Aux      : LocalON/OFF| x                    |
Messages : ResetAllCtr| o (121)              | Silence + Reinit of RMT routine
         : AllNotesOFF| o (123)              | Note OFF
         : ActiveSense| x                    |
         : SystemReset| o                    | Silence + Reinit of RMT routine
---------------------------------------------------------------------------
Mode 1: OMNI ON, POLY              Mode 2: OMNI ON, MONO              o: Yes
Mode 3: OMNI OFF, POLY             Mode 4: OMNI OFF, MONO             x: No
```

APPENDIX D

# Building from the source

RITMO is built with CMake and Qt. The full instructions are in the file `BUILD.md` of the source; here is the short version.

## D.1 Linux

```
sudo apt install cmake qt6-base-dev portaudio19-dev librtmidi-dev
cmake -B build-qt -DCMAKE_BUILD_TYPE=Release
cmake --build build-qt -j
./build-qt/out/ritmo song.rmt
```

The program is `build-qt/out/ritmo`, with the `resources` folder next to it. It needs Qt 6: Qt 5 is no longer supported from 2.6.

## D.2 Windows and macOS

Windows is built with MSYS2 (MinGW64: GCC, Qt 6, PortAudio, RtMidi) and macOS with the Apple compiler and the Qt of the official installer. The GitHub workflows of the repository (`.github/workflows`) build the packages of both, and the AppImage for Linux (`scripts/build-appimage.sh`), and are the best description of the steps.

## D.3 Linux riscv64 and ppc64

There is no AppImage for these architectures (no official Qt 6, no `linuxdeploy`). The workflow `.github/workflows/build-linux-ports.yml` builds the program in a Debian container of the architecture emulated by QEMU: `debian:trixie` for riscv64, a `debootstrap` of Debian ports *sid* for ppc64. Two scripts do the work, and can be started by hand in any Debian of the architecture:

```
docker run --rm --platform linux/riscv64 -v "$PWD":/src -w /src debian:trixie \
    sh -c 'scripts/build-tarball.sh && scripts/build-deb.sh'
```

`scripts/build-tarball.sh` makes the `.tar.gz` with the libraries (`lib`, `plugins`) and the launcher; `scripts/build-deb.sh` makes the `.deb`, whose dependencies come from `dpkg-shlibdeps`. Both run the program without a display as a test.

Every push tests the engine on PowerPC 64 big endian (`.github/workflows/test-bigendian.yml`): `RmtCoreTest` is built for x86_64 and cross-compiled for ppc64, run by `qemu-ppc64`, and `scripts/test-endian.sh` compares the POKEY registers, the sound and the screen of every song of `rmt/songs`.

## D.4 Core only and tests

`-DRMT_BUILD_CORE_ONLY=ON` builds the engine and the audio and MIDI backends without Qt, and with `-DRMT_CORE_TEST=ON` the `RmtCoreTest` program, a smoke test of the engine. The `scripts/test-scripting.sh` script runs the reference scripts.

## D.5 Test hooks

For tests and documentation the program can run without a display (`QT_QPA_PLATFORM=offscreen`). The environment variables `RMT_QT_GRAB` (save the window and quit), `RMT_QT_KEYS`, `RMT_QT_COMMANDS`, `RMT_QT_FILEDIALOG`, `RMT_QT_DIALOG` and `RMT_QT_MENU_GRAB` (the menus as images) press keys, run menu commands, answer the dialogs and save them; `RMT_QT_FONT` fixes the font of the menus and of the dialogs, so that the images do not depend on the desktop. The screenshots of this manual were taken that way, by `doc/manual/make-screenshots.sh`.

APPENDIX E

# Credits and licenses

## E.1 Credits

- Radek Štěrba, Raster/C.P.U., 2002–2009 (R.I.P.). The original RASTER Music Tracker. Thank you for everything you did; we truly miss you.
- Robert Petruzela (Bob!k/C.P.U.) and JirkaS/C.P.U.
- Vin Samuel (VinsCool), 2021–2024: RMT 1.31 to 1.34, the POKEY tuning calculations, the SAP-R dumper and the VU-Player, and much more.
- Peter Dell (JAC!), 2024 to present: RMT 1.35 and the base of this fork.
- Gianluca Renzi, 2026: the Qt port and the RITMO fork (Linux, Windows, macOS).

Also: the LZSS compression is by DMSC, ported to C++ by VinsCool; the unrolled LZSS music driver is by Rensoupp; the bitmap graphics, the ideas and the beta testing are by PG; advice on tuning from synthpopalooza and OPNA2608; ideas and suggestions from Enderdude, Spring, Ivop, Tatqoo and Miker. Thanks to Fox/Taquart (XASM and ASAP), Jaskier/Taquart (TMC and optimizations of the RMT routines), the people of the Chiptune Café, AtariAge and GBAtemp, the Polish Atarians of Atariarea and all the 8-bit Atarians of the world.

## E.2 The fonts

The logo and the icon of RITMO, and the cover of this manual, are drawn with the glyphs of the IBM VGA 9x16 font, in the version *PxPlus IBM VGA 9x16* of *The Ultimate Oldschool PC Font Pack* by VileR (`https://int10h.org/oldschool-pc-fonts/`), licensed under CC BY-SA 4.0.

## E.3 Disclaimer

RMT IS A SOFTWARE WITHOUT WARRANTY OF ANY KIND. THE AUTHOR DOES NOT WARRANT, GUARANTEE, OR MAKE ANY REPRESENTATIONS REGARDING THE USE, OR THE RESULTS OF USE OF THE SOFTWARE, OR WRITTEN MATERIALS, IN TERMS OF CORRECTNESS, ACCURACY, RELIABILITY, CURRENTNESS, OR OTHERWISE. THE ENTIRE RISK AS TO THE RESULTS AND PERFORMANCE OF THE SOFTWARE IS ASSUMED BY YOU.